Bharatiya Vidya Bhavan's

SARDAR PATEL INSTITUTE OF TECHNOLOGY

Department of Computer Engineering - Cryptography and Network Security Lab

Experiment 8

Evaluation of Classical and Post-Quantum Digital Signature Algorithms using Docker-Based Client-Server Communication

Field	Value
Name	Anusha Dalal
UID	2024300032
Date	16 October 2026
Algorithms	RSA-PSS (RSA-3072), ECDSA (P-256), ML-DSA-44, SLH-DSA-SHA2-128s

Aim

To implement the complete digital-signature workflow (key generation, signing, transmission, verification and tamper detection) between two Docker containers acting as sender and receiver, for two classical algorithms (RSA-PSS and ECDSA) and two NIST-standardised post-quantum algorithms (ML-DSA and SLH-DSA), and to compare experimentally their key sizes, signature sizes, speed and communication overhead, and to study why post-quantum signatures became necessary.

UID-based individualisation (Section 5)

Item	Value for UID 2024300032
Document line 1	Transfer Rs.3032 to Account 0032
Document line 2	Issued by UID: 2024300032
A = 1000 + (UID mod 9000)	1000 + 2032 = 3032
ECDSA private scalar d	d = (int(SHA-256(UID || "CNS-DSIG-ECDSA-KEY")) mod (n-1)) + 1 = 0x3225474890762ddbc056efc9926a6ed23cf2e7b72731af6acd57e5949ab4f561
Benchmark message	random.Random(2024300032).randbytes(1024), SHA-256 prefix c0aa6017dac0b92a
Tamper index (document, 59 B)	2024300032 mod 59 = 2
Tamper index (signatures)	ECDSA: mod 71 = 4, RSA-PSS: mod 384 = 256, ML-DSA: mod 2420 = 1492, SLH-DSA: mod 7856 = 5232

1. Part A - Hand-drawn digital-signature diagram

[ATTACH THE HAND-DRAWN SHEET HERE] - Part A must be drawn with pen/pencil and a ruler on a plain/ruled sheet before using the computer, with name, UID and date in the top-right corner, and appended to this document. The checklist below lists the eight required elements; a computer-drawn reference of the same diagram (to copy from) is given in Appendix A.

No.	Required element	Naming / convention used	Done
1	Two parties and an insecure channel	Sender (Alice) left, Receiver (Bob) right, channel between	[ ]
2	Message and hash	Message M; hash function H( ); digest h = H(M)	[ ]
3	Sender's key pair	Private key SK_A (secret, dashed arrow), public key PK_A	[ ]
4	Signing step	S = Sign(SK_A, h); output pair (M || S)	[ ]
5	Public-key distribution	Arrow PK_A to receiver; SK_A is never sent	[ ]
6	Receiver side	Received M' and S; h' = H(M'); Verify(PK_A, h', S)	[ ]
7	Decision	Decision box: ACCEPT (valid) / REJECT (invalid)	[ ]
8	Attacker case	Mallory modifies M to M' in the channel -> REJECT	[ ]

2. Experiment architecture and Docker environment (Parts B and C)

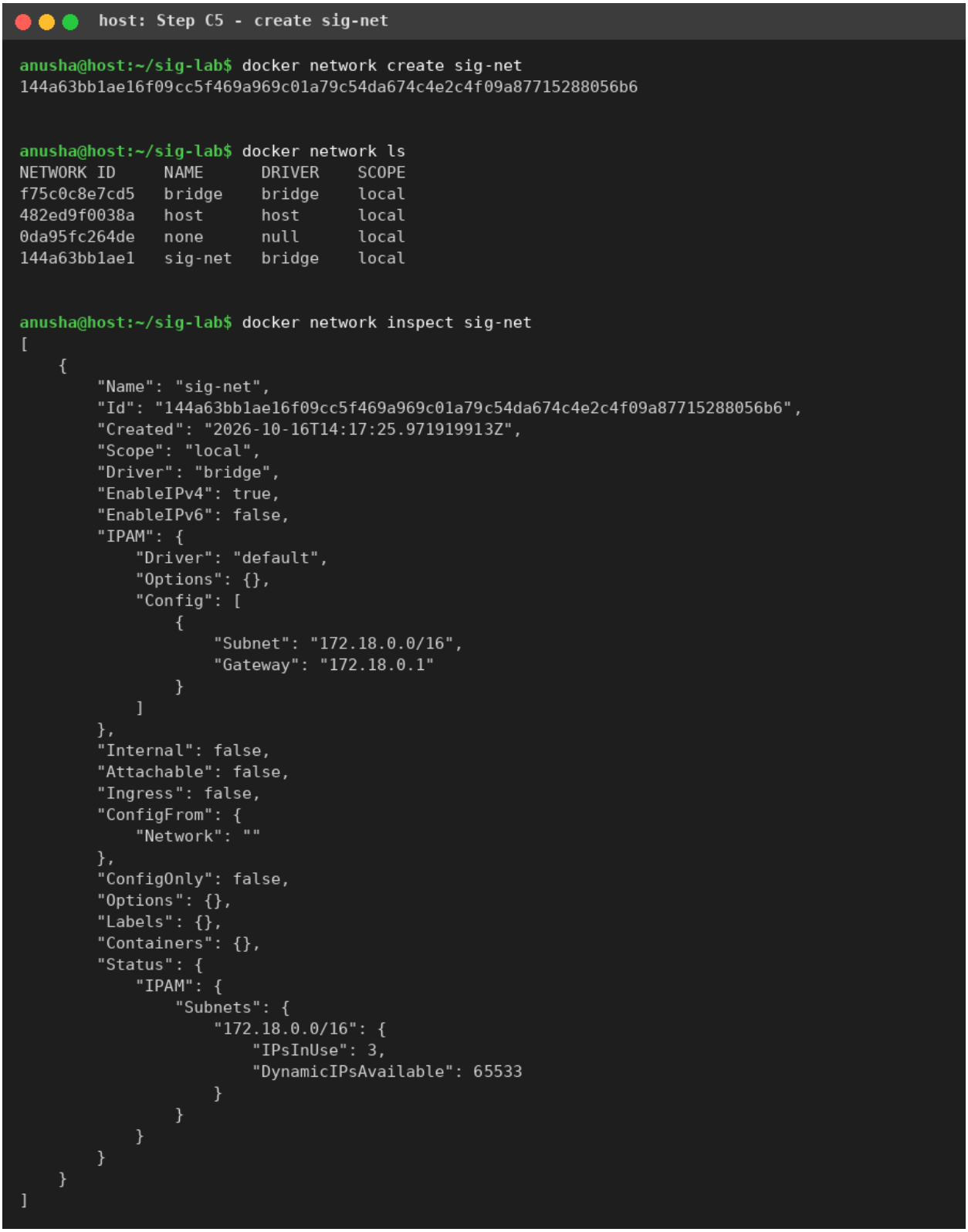
One image (sig-lab:1.0) and two containers, sender (Alice) and receiver (Bob), attached to the user-defined bridge network sig-net. The host folders sender/ and receiver/ are bind-mounted at /lab. The sender generates keys, signs the document and serves outbox/ over HTTP on port 8000; the receiver downloads document, signature and public key with curl and verifies them. Only public information crosses the network; private keys stay in the sender's keys/ folder.

IP addresses (Step C7): sender = 172.18.0.2, receiver = 172.18.0.3 (subnet 172.18.0.0/16 of sig-net).

Table 1: Execution environment

CPU model	Cores / threads	RAM	Host OS	Docker version	cryptography version	liboqs version
Intel(R) Xeon(R) Processor @ 2.80GHz	4 vCPUs (4 threads)	15 GiB	Ubuntu 24.04.5 LTS, kernel 6.18.44	29.8.2 (build 7fc2dff)	50.0.2	liboqs 0.16.0 (liboqs-python 0.16.0.1)

Container: Ubuntu 24.04.5 LTS, Python 3.12.3. The machine is a 4-vCPU cloud virtual machine. Note: the build machine reaches the Internet through a TLS-inspecting proxy, so the image was built from a copy of the Dockerfile that additionally trusts that proxy's CA certificate (Dockerfile.sandbox in the repository); the submitted Dockerfile is exactly the one in Step C3.



Screenshot 1: Step C5 - docker network create / ls / inspect sig-net

```
host: Step C6 - start containers / docker ps

anusha@host:~/sig-lab$ export STUDENT_UID=2024300032


anusha@host:~/sig-lab$ docker run -dit --name sender   --hostname sender   --network sig-net -e STUDENT_UID=2024300032 -v /home/user/cns8/sender:/lab
  sig-lab:1.0
26c3c6c0a3dfa5459e45d83f4b69c814c99f2997e96dd3e0abd26dce34c4216e


anusha@host:~/sig-lab$ docker run -dit --name receiver --hostname receiver --network sig-net -e STUDENT_UID=2024300032 -v /home/user/cns8/receiver:/la
b sig-lab:1.0
479e1892431dea47068dc938d27c4de070dcbfa620ec9c71485f6351f3a98440

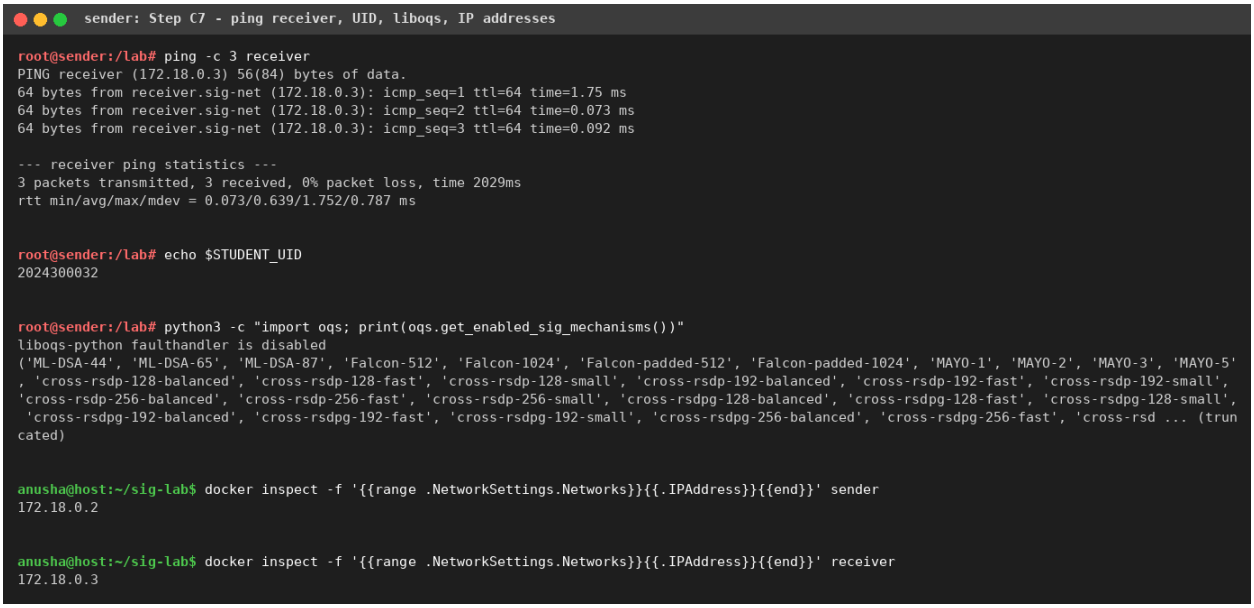

anusha@host:~/sig-lab$ docker ps
CONTAINER ID   IMAGE         COMMAND            CREATED                  STATUS                  PORTS     NAMES
479e1892431d   sig-lab:1.0   "sleep infinity"   Less than a second ago   Up Less than a second             receiver
26c3c6c0a3df   sig-lab:1.0   "sleep infinity"   Less than a second ago   Up Less than a second             sender
```

Screenshot 2: Step C6 - both containers started and Up (docker ps)

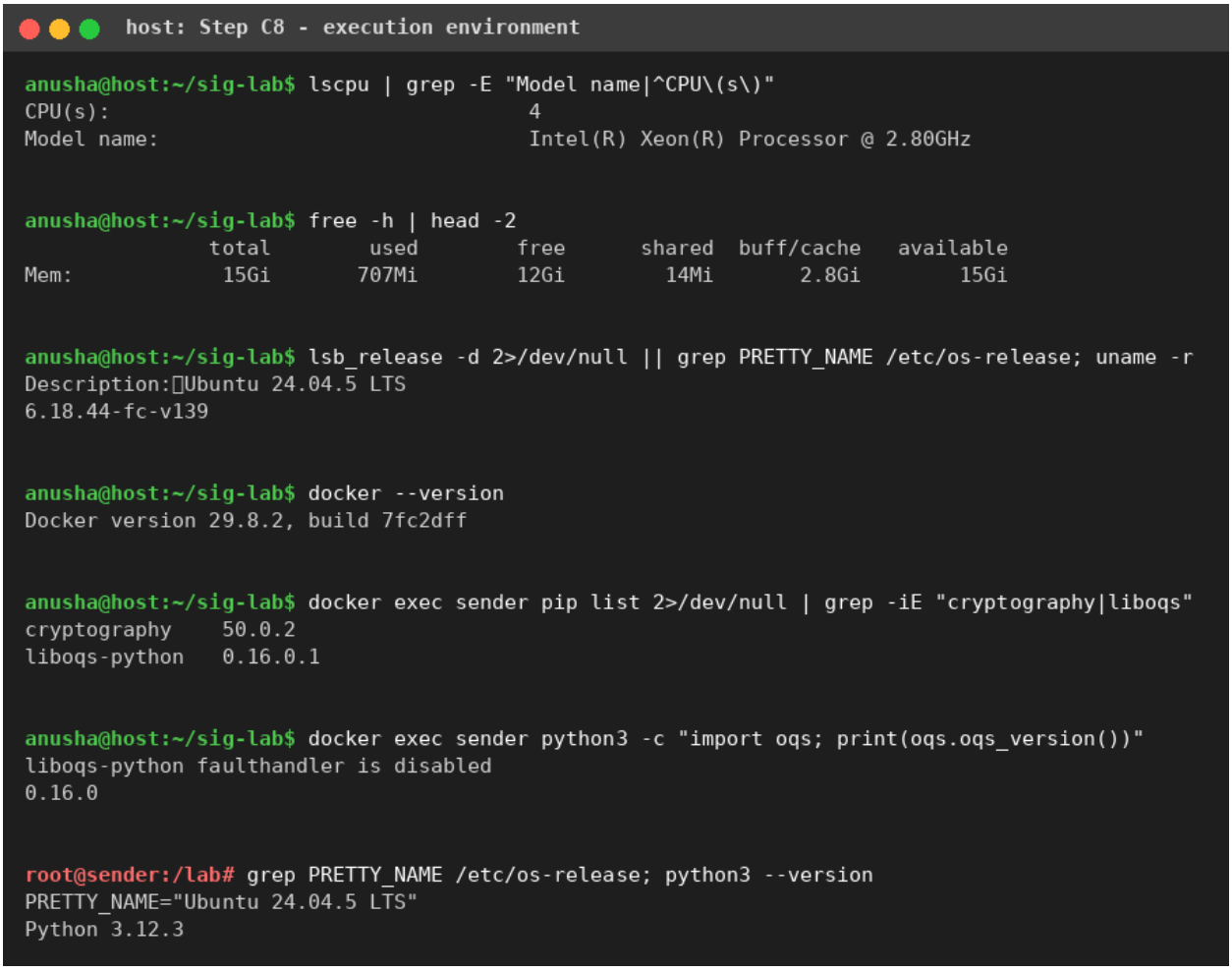
```
host: Step C7 - docker network inspect sig-net

anusha@host:~/sig-lab$ docker network inspect sig-net
[
    {
        "Name": "sig-net",
        "Id": "144a63bb1ae16f09cc5f469a969c01a79c54da674c4e2c4f09a87715288056b6",
        "Created": "2026-10-16T14:17:25.971919913Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {},
        "Labels": {},
        "Containers": {
            "26c3c6c0a3dfa5459e45d83f4b69c814c99f2997e96dd3e0abd26dce34c4216e": {
                "Name": "sender",
                "EndpointID": "e2670e9c06aa1e248d73c5515dadecd49d22bc802fc395dd0d035a39ba6ca7e6",
                "MacAddress": "be:b3:ac:3b:b5:fb",
                "IPv4Address": "172.18.0.2/16",
                "IPv6Address": ""
            },
            "479e1892431dea47068dc938d27c4de070dcbfa620ec9c71485f6351f3a98440": {
                "Name": "receiver",
                "EndpointID": "4088899ba7d6d6c4c706b7ba642726aaa1139ed3ca7335b26c86cdf92a85a1dd",
                "MacAddress": "82:47:d7:a2:7c:dc",
                "IPv4Address": "172.18.0.3/16",
                "IPv6Address": ""
            }
        },
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.18.0.0/16": {
                        "IPsInUse": 5,
                        "DynamicIPsAvailable": 65531
                    }
                }
            }
        }
    }
]
```

Screenshot 3: Step C7 - docker network inspect sig-net (both containers attached)

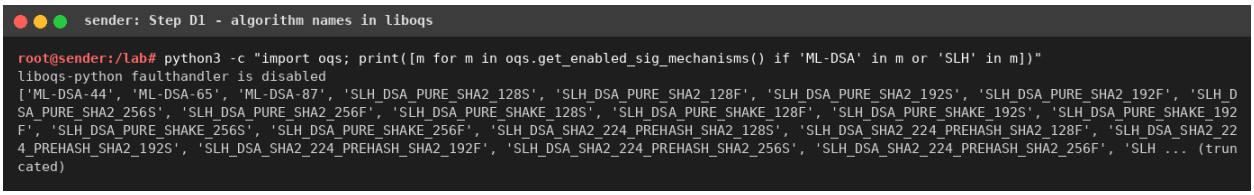


Screenshot 4: Step C7 - ping test sender -> receiver, STUDENT_UID, liboqs mechanisms, IP addresses

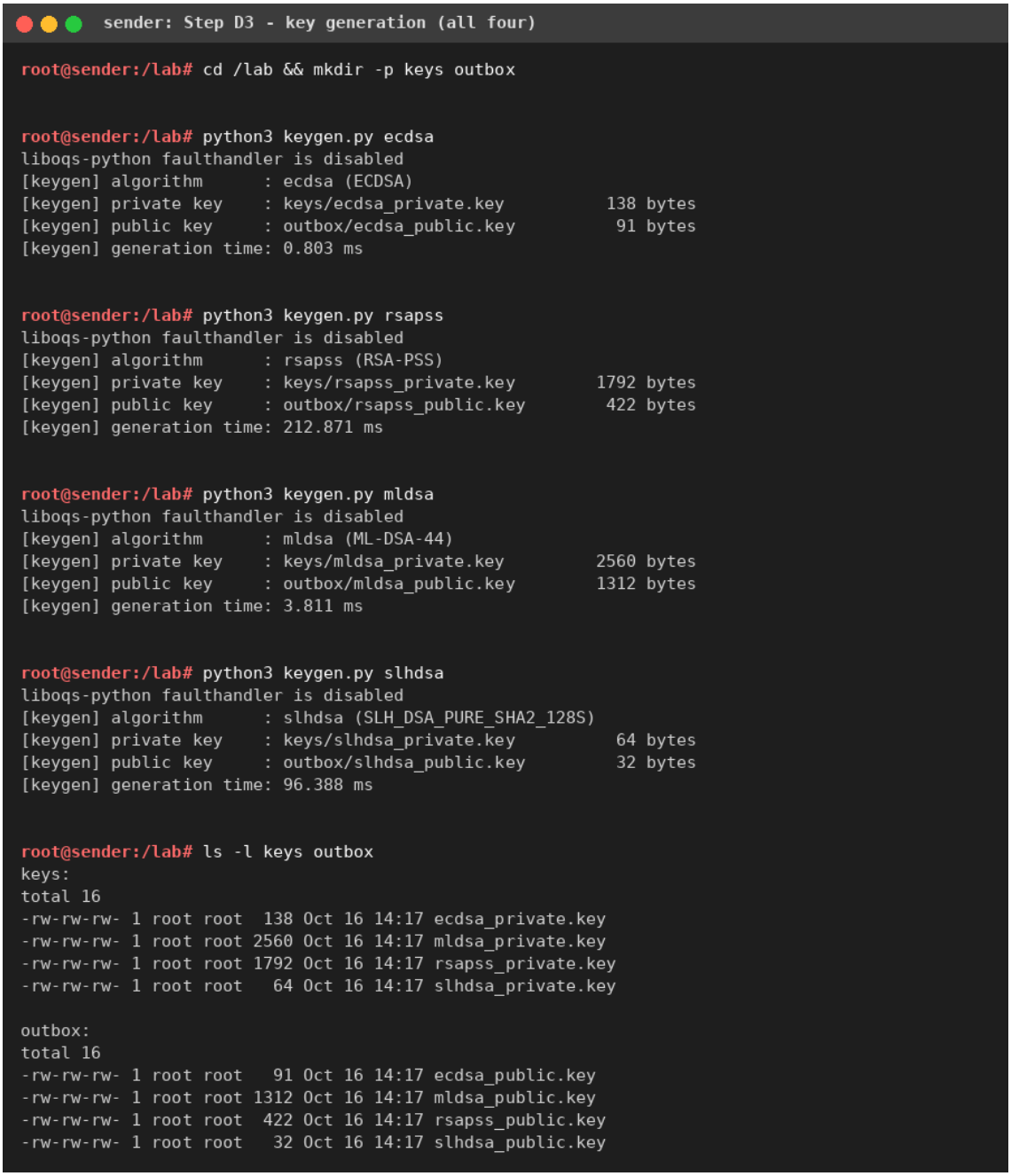


Screenshot 5: Step C8 - execution environment

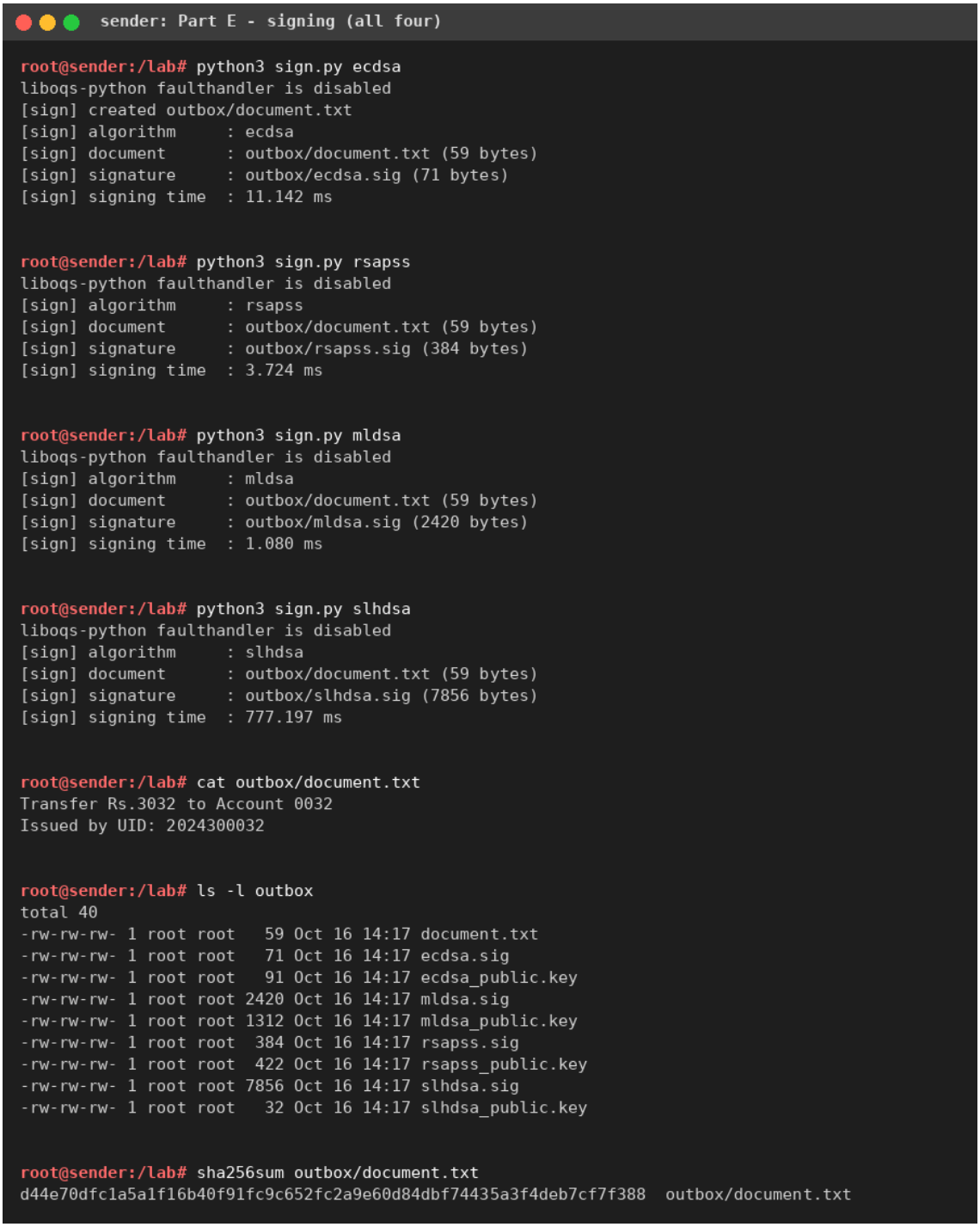
3. Key generation, signing, transfer and verification (Parts D-G)



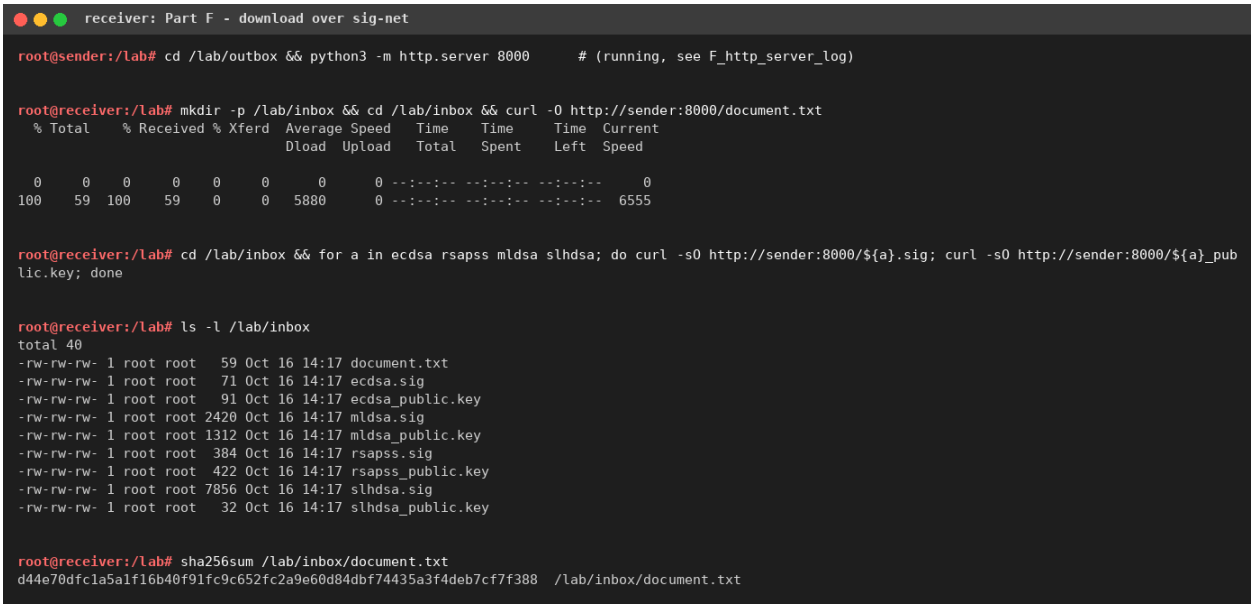
Screenshot 6: Step D1 - exact algorithm names: ML-DSA-44 and SLH_DSA_PURE_SHA2_128S



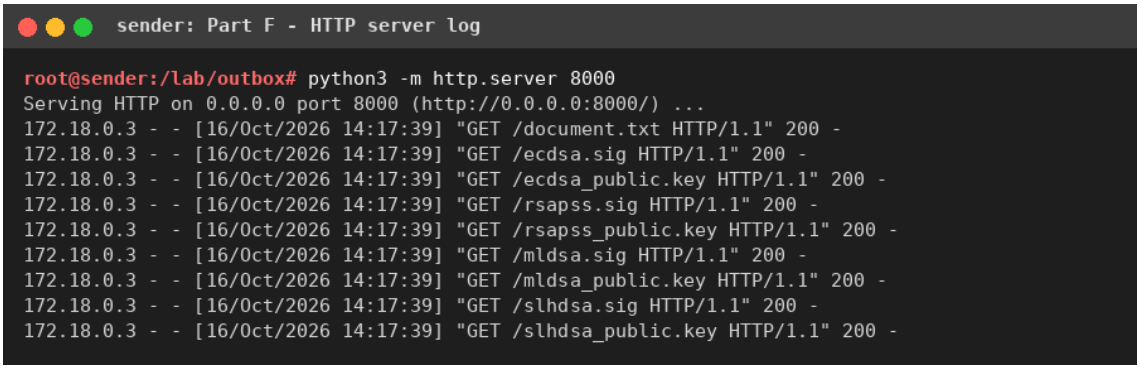
Screenshot 7: Step D3 - keygen.py for all four algorithms



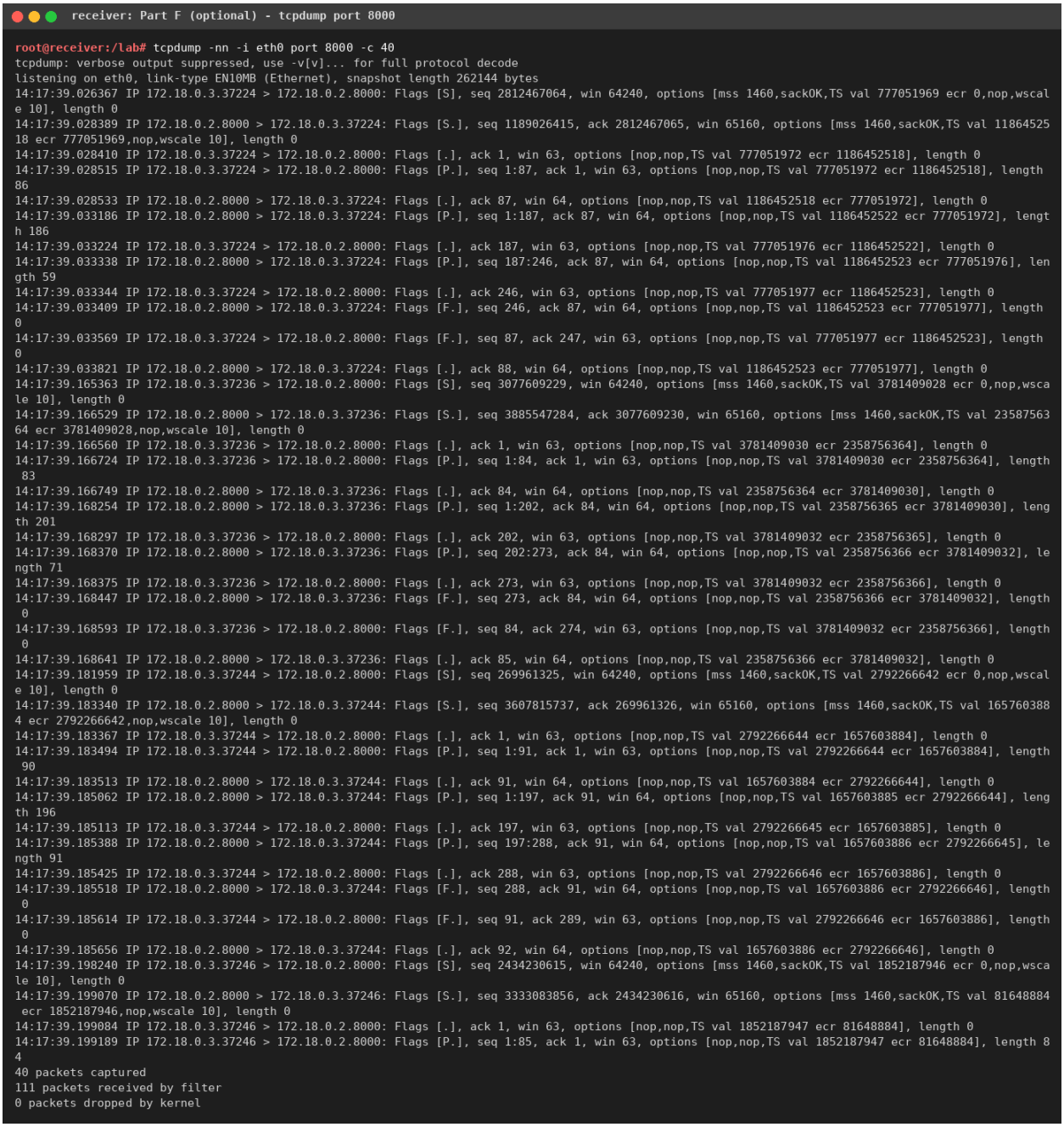
Screenshot 8: Part E - sign.py for all four algorithms, document and its SHA-256



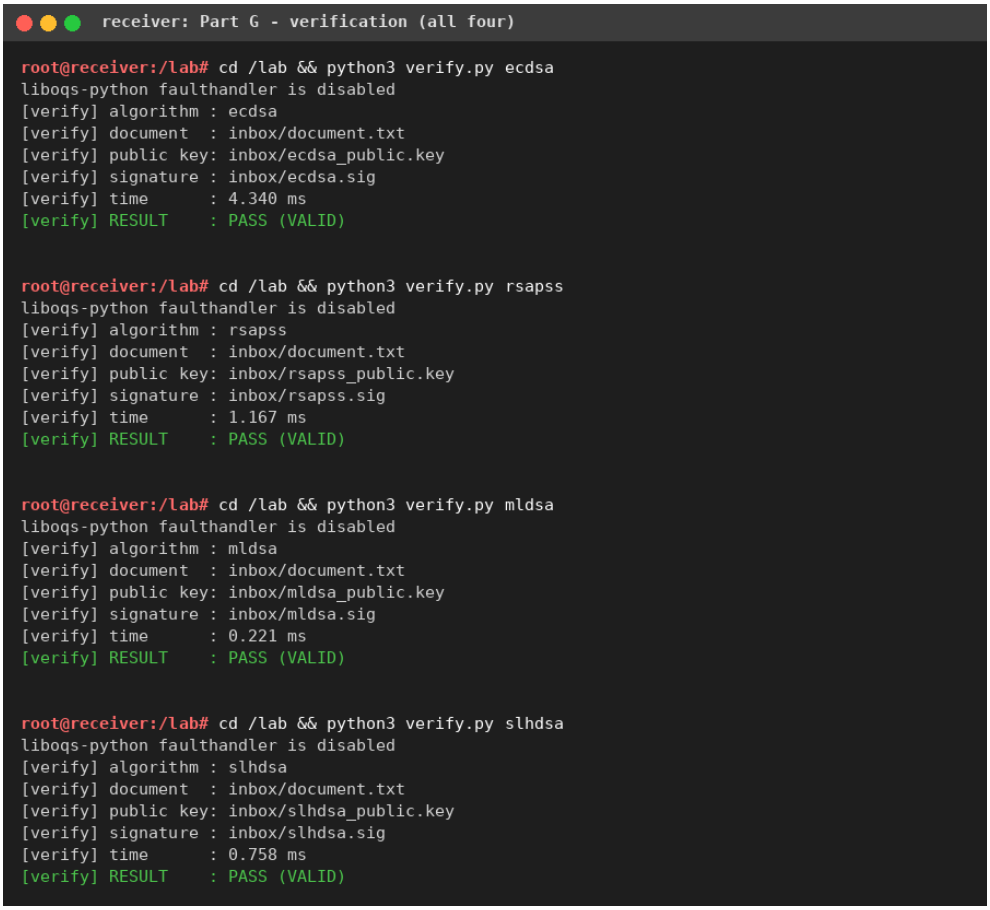
Screenshot 9: Part F - receiver downloads over sig-net; SHA-256 equals the sender's value



Screenshot 10: Part F - sender's HTTP server log; every request comes from the receiver's IP 172.18.0.3



Screenshot 11: Part F (optional) - tcpdump on the receiver, port 8000



Screenshot 12: Part G - verify.py prints PASS for all four algorithms

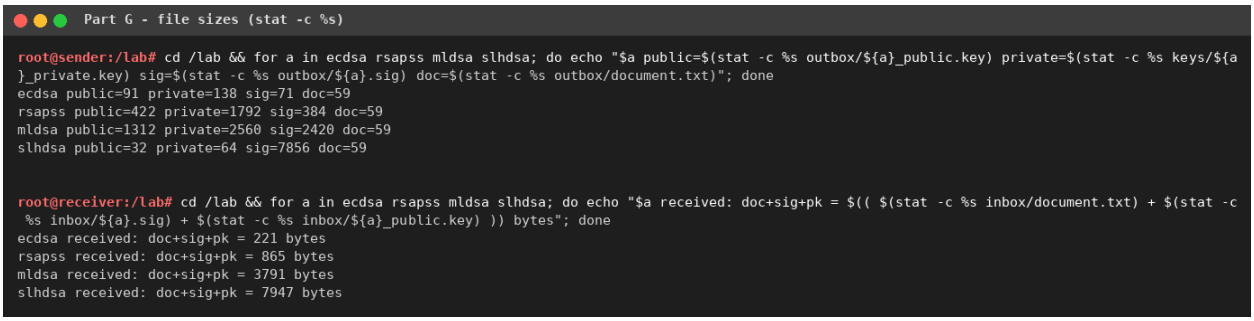
Document SHA-256 on the sender and the receiver: d44e70dfc1a5a1f16b40f91fc9c652fc2a9e60d84dbf74435a3f4deb7cf7f388 (identical, so the file was transferred unchanged).

4. Results: sizes, tampering and timings

Table 2: Key, signature and transmission sizes (bytes)

Algorithm	Public key	Private key	Signature	Document	Transmitted (doc + sig + public key)
ECDSA	91	138	71	59	221
RSA-PSS	422	1792	384	59	865
ML-DSA	1312	2560	2420	59	3791
SLH-DSA	32	64	7856	59	7947

Measured with stat -c %s (Screenshot 13). Sanity check: ML-DSA-44 public key 1312, secret key 2560, signature 2420 bytes and SLH-DSA-SHA2-128s public key 32, secret key 64, signature 7856 bytes - exactly the FIPS 204/205 values, since liboqs stores raw keys. The classical keys are DER encoded and therefore slightly larger than the raw values (P-256: 65-byte point + 26-byte SubjectPublicKeyInfo header = 91 B; ECDSA signatures are DER and vary between 70 and 72 B, 71 B here; an RSA-3072 PKCS#8 key varies by about 1 B between key pairs, e.g. 1792 B here and 1793 B in the benchmark).



Screenshot 13: file sizes on the sender and bytes received by the receiver

Table 3: Verification results

Test case	ECDSA	RSA-PSS	ML-DSA	SLH-DSA	Expected
1. Original document + original signature	PASS	PASS	PASS	PASS	PASS
2. One character of document changed	FAIL	FAIL	FAIL	FAIL	FAIL
3. One bit of signature changed	FAIL	FAIL	FAIL	FAIL	FAIL
4. Correct document + signature, wrong public key (receiver's own key pair)	PASS	FAIL	FAIL	FAIL	FAIL
4b. (supplementary) ECDSA public key derived from a different UID (2024300033)	FAIL	n/a	n/a	n/a	FAIL

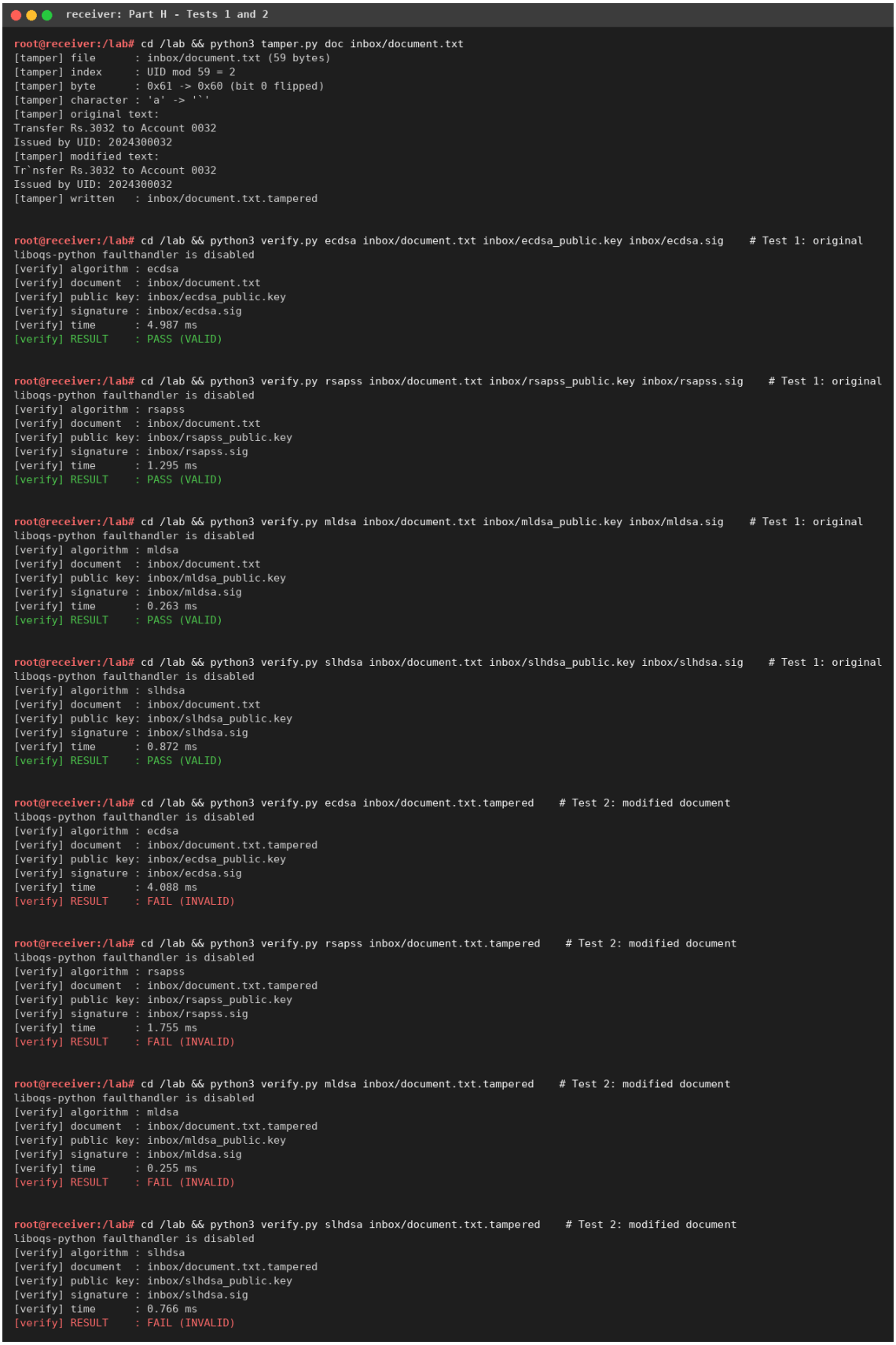
Original and modified document (Test 2). index = UID mod 59 = 2, byte 0x61 ('a') -> 0x60 ('`'):

	Document text
Original M	Transfer Rs.3032 to Account 0032 / Issued by UID: 2024300032
Modified M'	Tr`nsfer Rs.3032 to Account 0032 / Issued by UID: 2024300032

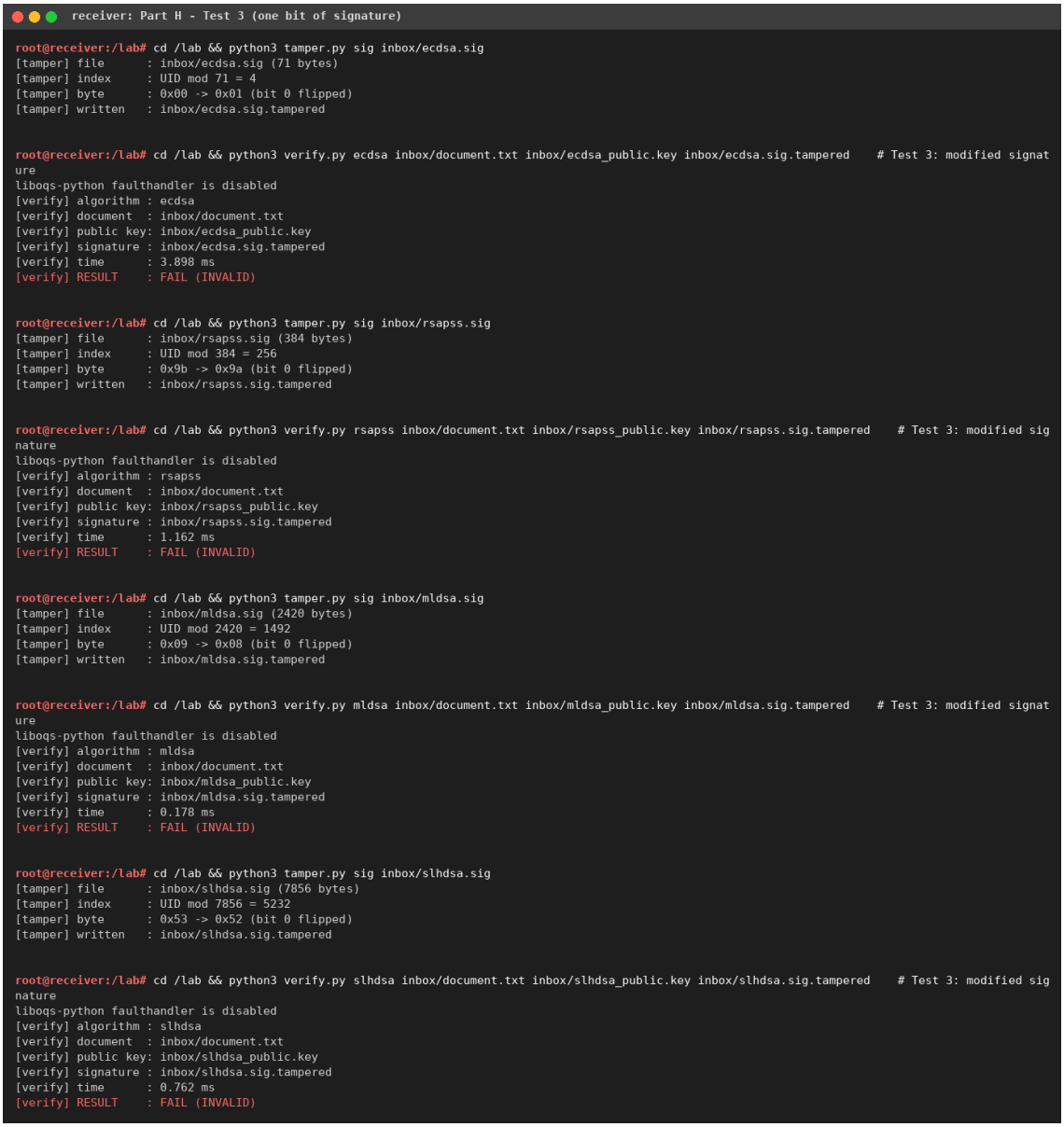
Signature bit flipped (Test 3) at index UID mod length: ECDSA byte 4 (0x00 -> 0x01), RSA-PSS byte 256 (0x9b -> 0x9a), ML-DSA byte 1492 (0x09 -> 0x08), SLH-DSA byte 5232 (0x53 -> 0x52).

Observation on Test 4 (ECDSA = PASS). In Test 4 the receiver runs keygen.py ecdsa. The ECDSA private key is not random: as required by Section 5 it is derived from the UID, and both containers receive the same STUDENT_UID. The receiver therefore regenerates exactly the same key pair as the sender (cmp confirms the

two ecdsa_public.key files are byte-identical, Screenshot 16), so the "wrong" key is in fact the right one and verification correctly passes. This is precisely the weakness of the warning box in Section 5: anyone who knows (or brute-forces, ~33 bits) the UID can recreate the private key and forge signatures. RSA-PSS, ML-DSA and SLH-DSA keys come from the secure random generator, so the receiver's key pairs differ and verification fails as expected. With a genuinely different ECDSA key (Test 4b, key derived from UID 2024300033) ECDSA also returns FAIL.



Screenshot 14: Part H - tamper.py doc, Tests 1 and 2



Screenshot 15: Part H - Test 3 (one bit of each signature flipped)



Screenshot 16: Part H - Test 4 (receiver's own key pair) and supplementary Test 4b

Table 4: Timing results (milliseconds, mean ± standard deviation)

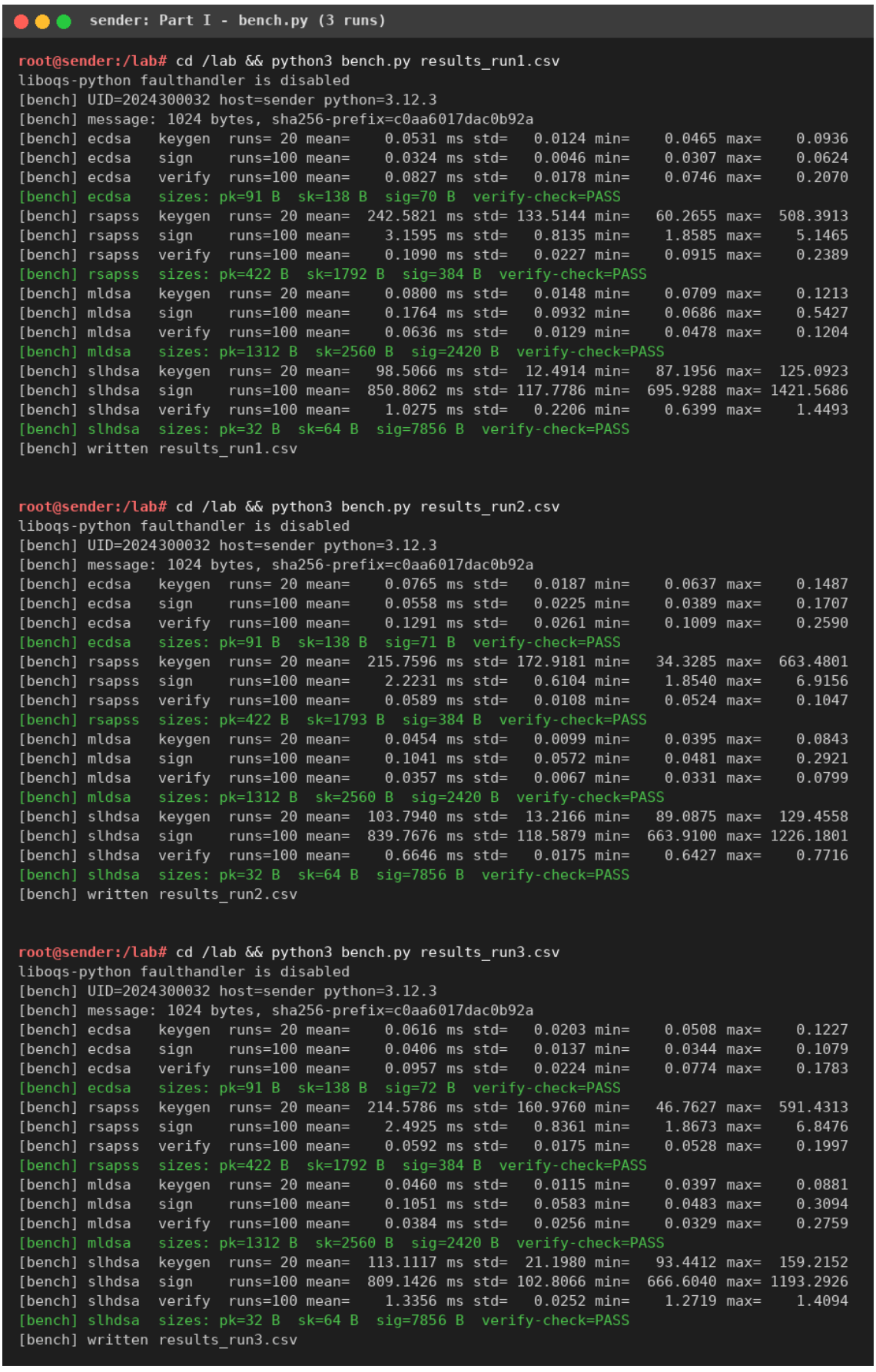
Sender container, median of three bench.py runs (run 2; runs ranked by total mean time). Each value is the mean ± standard deviation over the individual calls: key generation n = 20, signing n = 100, verification n = 100, after 5 warm-up calls; 1024-byte UID-seeded message.

Algorithm	Key generation	Signing	Verification	Sign / Verify ratio
ECDSA	0.0765 ± 0.0187	0.0558 ± 0.0225	0.1291 ± 0.0261	0.43
RSA-PSS	215.7596 ± 172.9181	2.2231 ± 0.6104	0.0589 ± 0.0108	37.74
ML-DSA	0.0454 ± 0.0099	0.1041 ± 0.0572	0.0357 ± 0.0067	2.92
SLH-DSA	103.7940 ± 13.2166	839.7676 ± 118.5879	0.6646 ± 0.0175	1263.57

Supplementary - variation between the three runs on the sender (mean of the three run means ± their standard deviation), the receiver container, and the sender limited to one CPU (docker update --cpus 1):

Alg.	KeyGen (3 runs)	Sign (3 runs)	Verify (3 runs)	KeyGen recv.	Sign recv.	Verify recv.	Sign --cpus 1
ECDSA	0.0637 ± 0.0118	0.0429 ± 0.0119	0.1025 ± 0.0239	0.0908	0.0559	0.1522	0.0347
RSA-PSS	224.3068 ± 15.8379	2.6250 ± 0.4821	0.0757 ± 0.0288	258.2299	2.6541	0.0875	2.3058
ML-DSA	0.0571 ± 0.0198	0.1285 ± 0.0415	0.0459 ± 0.0154	0.0471	0.0954	0.0365	0.1082
SLH-DSA	105.1374 ± 7.3946	833.2388 ± 21.5855	1.0092 ± 0.3359	110.9689	786.0777	0.6878	797.9140

The sender and receiver containers give the same picture (same image, same host): differences are within run-to-run noise. Limiting the container to one CPU changes little because each operation is single-threaded. RSA key generation shows a very large standard deviation because the time to find two random 1536-bit primes is itself random.



Screenshot 17: Part I - bench.py on the sender (three runs)

receiver: Part I - bench.py

```
root@receiver:/lab# cd /lab && python3 bench.py results.csv
liboqs-python faulthandler is disabled
[bench] UID=2024300032 host=receiver python=3.12.3
[bench] message: 1024 bytes, sha256-prefix=c0aa6017dac0b92a
[bench] ecdsa   keygen  runs= 20 mean=    0.0908 ms std=   0.0204 min=    0.0787 max=    0.1586
[bench] ecdsa   sign    runs=100 mean=    0.0559 ms std=   0.0136 min=    0.0453 max=    0.1295
[bench] ecdsa   verify  runs=100 mean=    0.1522 ms std=   0.0246 min=    0.1016 max=    0.2678
[bench] ecdsa   sizes: pk=91 B  sk=138 B  sig=71 B  verify-check=PASS
[bench] rsapss  keygen  runs= 20 mean=  258.2299 ms std= 166.1175 min=   75.0116 max=  620.7739
[bench] rsapss  sign    runs=100 mean=    2.6541 ms std=   0.7561 min=    1.8775 max=    4.3344
[bench] rsapss  verify  runs=100 mean=    0.0875 ms std=   0.1650 min=    0.0527 max=    1.6344
[bench] rsapss  sizes: pk=422 B  sk=1793 B  sig=384 B  verify-check=PASS
[bench] mldsa   keygen  runs= 20 mean=    0.0471 ms std=   0.0108 min=    0.0391 max=    0.0867
[bench] mldsa   sign    runs=100 mean=    0.0954 ms std=   0.0479 min=    0.0474 max=    0.2459
[bench] mldsa   verify  runs=100 mean=    0.0365 ms std=   0.0087 min=    0.0329 max=    0.1066
[bench] mldsa   sizes: pk=1312 B  sk=2560 B  sig=2420 B  verify-check=PASS
[bench] slhdsa  keygen  runs= 20 mean=  110.9689 ms std=  17.3681 min=   91.5047 max=  150.8713
[bench] slhdsa  sign    runs=100 mean=  786.0777 ms std=  96.0449 min=  663.8267 max= 1360.1079
[bench] slhdsa  verify  runs=100 mean=    0.6878 ms std=   0.0325 min=    0.6637 max=    0.8916
[bench] slhdsa  sizes: pk=32 B  sk=64 B  sig=7856 B  verify-check=PASS
[bench] written results.csv
```

Screenshot 18: Part I - bench.py on the receiver

sender: Part I (optional) - --cpus 1 + median run

```
anusha@host:~/sig-lab$ docker update --cpus 1 sender
sender


root@sender:/lab# cd /lab && python3 bench.py results_cpus1.csv
liboqs-python faulthandler is disabled
[bench] UID=2024300032 host=sender python=3.12.3
[bench] message: 1024 bytes, sha256-prefix=c0aa6017dac0b92a
[bench] ecdsa   keygen  runs= 20 mean=    0.0688 ms std=   0.0449 min=    0.0467 max=    0.2308
[bench] ecdsa   sign    runs=100 mean=    0.0347 ms std=   0.0135 min=    0.0308 max=    0.1320
[bench] ecdsa   verify  runs=100 mean=    0.1277 ms std=   0.3720 min=    0.0747 max=    3.7654
[bench] ecdsa   sizes: pk=91 B  sk=138 B  sig=72 B  verify-check=PASS
[bench] rsapss  keygen  runs= 20 mean=  181.8936 ms std=  74.6825 min=   34.2512 max=  335.5321
[bench] rsapss  sign    runs=100 mean=    2.3058 ms std=   0.4168 min=    1.8793 max=    3.4427
[bench] rsapss  verify  runs=100 mean=    0.0573 ms std=   0.0143 min=    0.0525 max=    0.1699
[bench] rsapss  sizes: pk=422 B  sk=1793 B  sig=384 B  verify-check=PASS
[bench] mldsa   keygen  runs= 20 mean=    0.0471 ms std=   0.0159 min=    0.0394 max=    0.1033
[bench] mldsa   sign    runs=100 mean=    0.1082 ms std=   0.0541 min=    0.0477 max=    0.3089
[bench] mldsa   verify  runs=100 mean=    0.0361 ms std=   0.0108 min=    0.0328 max=    0.1028
[bench] mldsa   sizes: pk=1312 B  sk=2560 B  sig=2420 B  verify-check=PASS
[bench] slhdsa  keygen  runs= 20 mean=  111.2466 ms std=  17.6003 min=   88.6214 max=  143.8433
[bench] slhdsa  sign    runs=100 mean=  797.9140 ms std=  88.4552 min=  666.8458 max= 1062.5742
[bench] slhdsa  verify  runs=100 mean=    0.7118 ms std=   0.0595 min=    0.6604 max=    1.0155
[bench] slhdsa  sizes: pk=32 B  sk=64 B  sig=7856 B  verify-check=PASS
[bench] written results_cpus1.csv

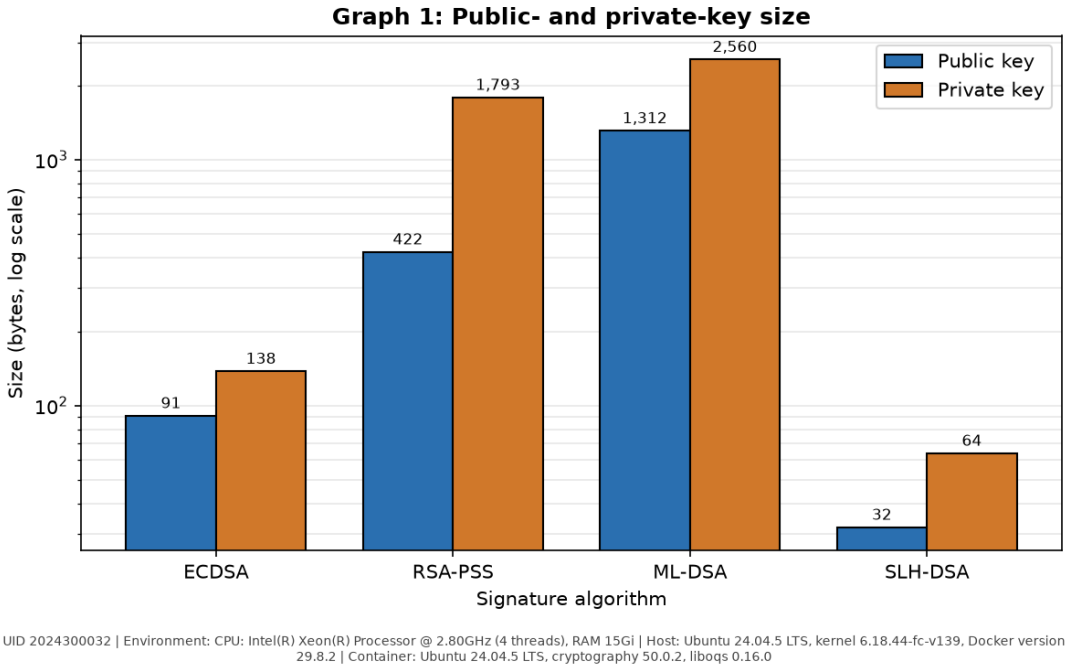

anusha@host:~/sig-lab$ docker update --cpus 0 sender
sender

total mean time per run (ms): {1: 1196.68, 2: 1162.71, 3: 1141.11} -> median run: 2

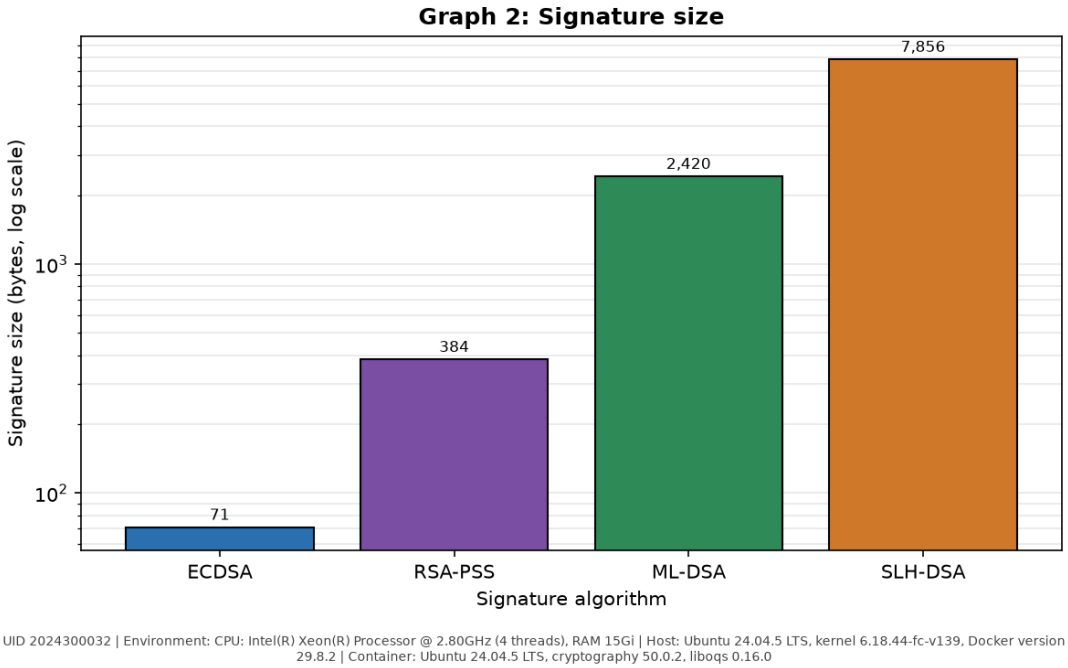
root@sender:/lab# cat results.csv
uid,algorithm,operation,runs,mean_ms,std_ms,min_ms,max_ms,public_key_bytes,private_key_bytes,signature_bytes
2024300032,ecdsa,keygen,20,0.0765,0.0187,0.0637,0.1487,91,138,71
2024300032,ecdsa,sign,100,0.0558,0.0225,0.0389,0.1707,91,138,71
2024300032,ecdsa,verify,100,0.1291,0.0261,0.1009,0.2590,91,138,71
2024300032,rsapss,keygen,20,215.7596,172.9181,34.3285,663.4801,422,1793,384
2024300032,rsapss,sign,100,2.2231,0.6104,1.8540,6.9156,422,1793,384
2024300032,rsapss,verify,100,0.0589,0.0108,0.0524,0.1047,422,1793,384
2024300032,mldsa,keygen,20,0.0454,0.0099,0.0395,0.0843,1312,2560,2420
2024300032,mldsa,sign,100,0.1041,0.0572,0.0481,0.2921,1312,2560,2420
2024300032,mldsa,verify,100,0.0357,0.0067,0.0331,0.0799,1312,2560,2420
2024300032,slhdsa,keygen,20,103.7940,13.2166,89.0875,129.4558,32,64,7856
2024300032,slhdsa,sign,100,839.7676,118.5879,663.9100,1226.1801,32,64,7856
2024300032,slhdsa,verify,100,0.6646,0.0175,0.6427,0.7716,32,64,7856
```

Screenshot 19: Part I (optional) - sender limited to 1 CPU, median-run selection

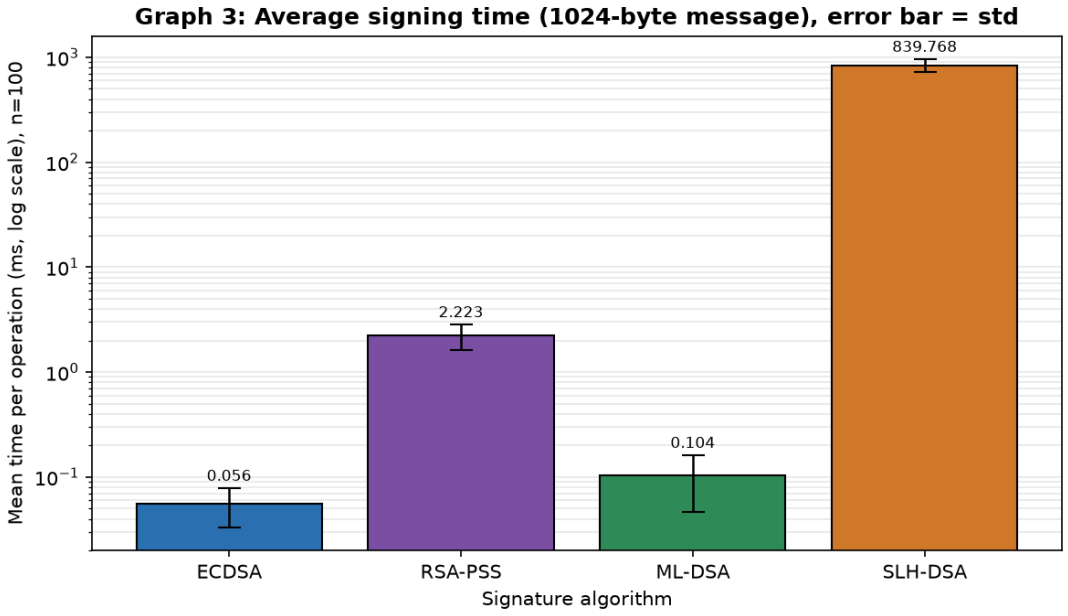
5. Graphical analysis (Section 15)



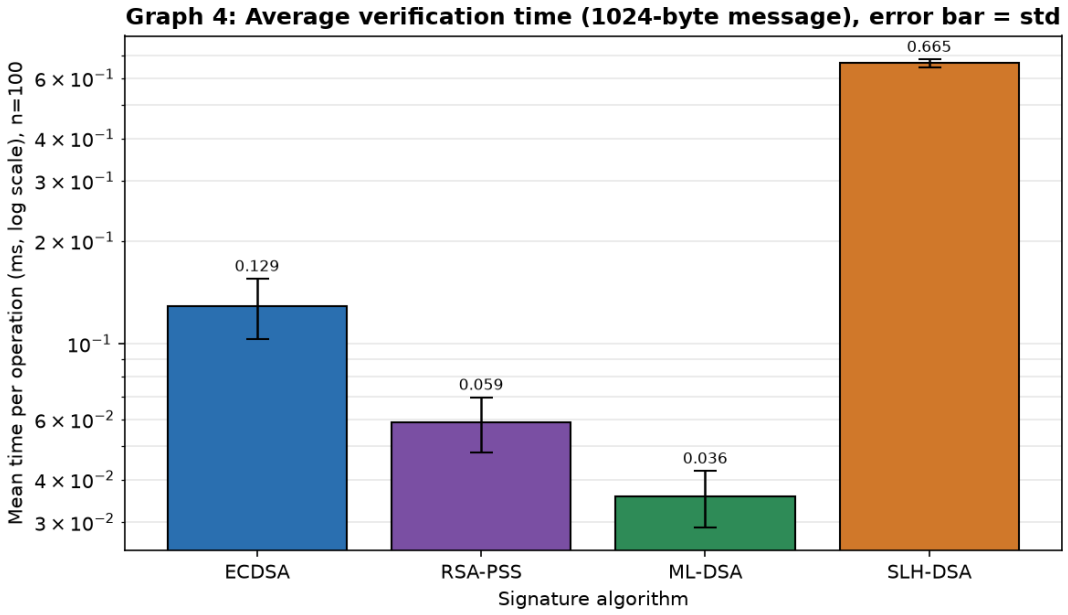
Graph 1: Public-key and private-key size (bytes, log scale). Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



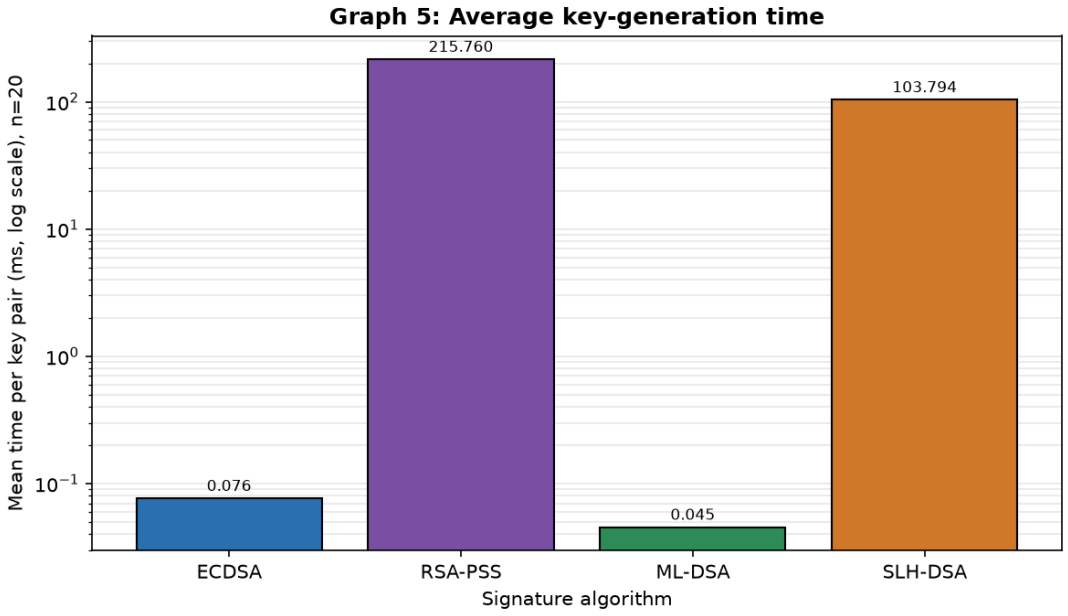
Graph 2: Signature size (bytes, log scale). Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



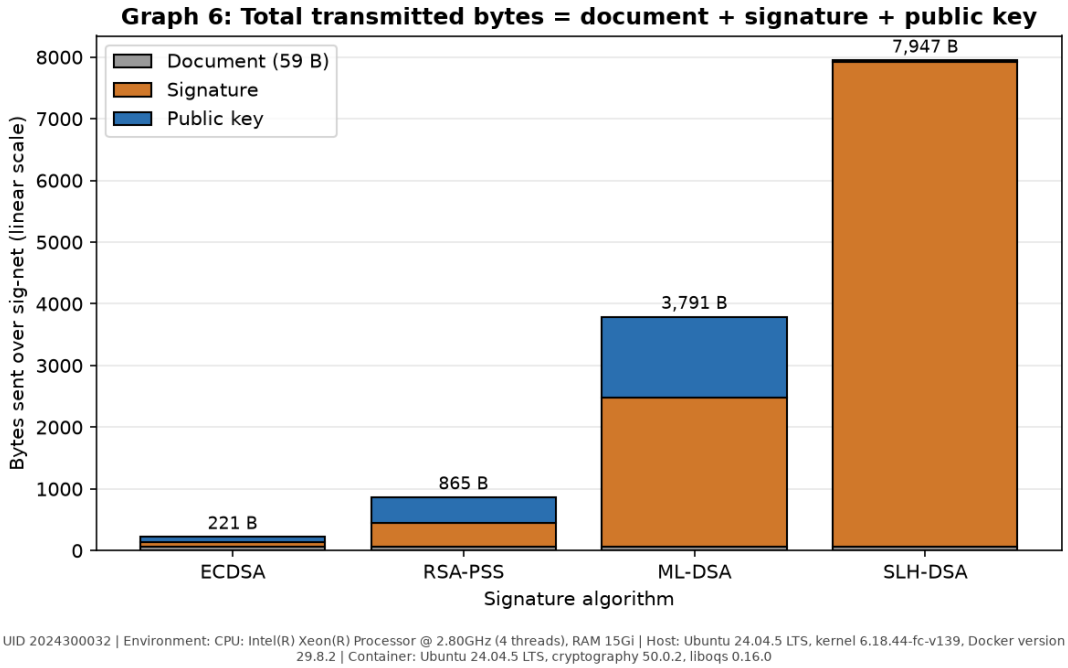
Graph 3: Average signing time (ms, log scale), error bar = standard deviation, n = 100. Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



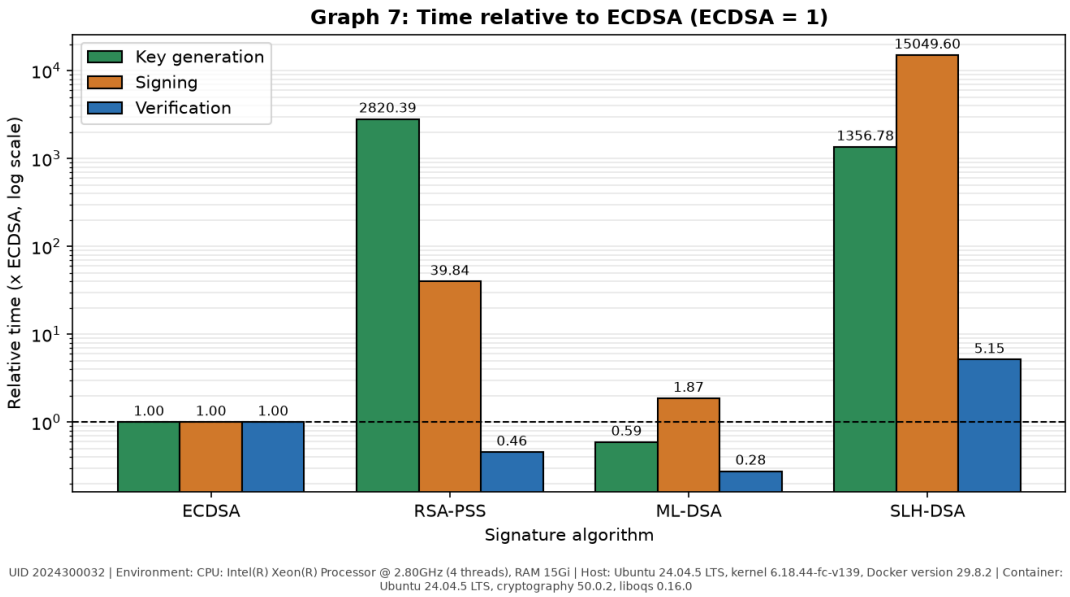
Graph 4: Average verification time (ms, log scale), error bar = standard deviation, n = 100. Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



Graph 5: Average key-generation time (ms, log scale), n = 20. Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



Graph 6: Total transmitted bytes = document + signature + public key (stacked). Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



Graph 7 (extra): Key-generation, signing and verification time relative to ECDSA (ECDSA = 1, log scale). Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.

6. Comparative analysis (Part J)

Table 5: Relative comparison with ECDSA as the baseline

Parameter	ECDSA	RSA-PSS	ML-DSA	SLH-DSA
Public key size	1.0	4.64	14.42	0.35
Signature size	1.0	5.41	34.08	110.65
Signing time	1.0	39.84	1.87	15049.60
Verification time	1.0	0.46	0.28	5.15
Key generation time	1.0	2820.39	0.59	1356.78
Total transmitted bytes	1.0	3.91	17.15	35.96

Table 6: Property comparison

Property	ECDSA	RSA-PSS	ML-DSA	SLH-DSA
Standard	FIPS 186-5	RFC 8017, FIPS 186-5	FIPS 204 (2024)	FIPS 205 (2024)
Security parameter / category	P-256, ~128-bit classical	RSA-3072, ~128-bit classical	ML-DSA-44, NIST category 2	SLH-DSA-SHA2-128s, NIST category 1
Mathematical basis	Elliptic-curve discrete logarithm	Integer factorisation	Module lattices (Module-LWE / Module-SIS)	Hash functions only (hypertree of XMSS/WOTS+ and FORS trees)
Resistant to Shor's algorithm?	No	No	Yes	Yes
Public key size	Small (91 B)	Medium (422 B)	Large (1312 B)	Very small (32 B)
Signature size	Small (71 B)	Medium (384 B)	Large (2420 B)	Very large (7856 B)
Fastest operation	Signing (0.056 ms)	Verification (0.059 ms)	Verification (0.036 ms)	Verification (0.665 ms)
Slowest operation	Verification (0.129 ms)	Key generation (215.8 ms)	Signing (0.104 ms)	Signing (839.8 ms)
Maturity / deployment	Very mature; TLS, SSH, code signing, smart cards, cryptocurrencies	Very mature (decades); PKI, TLS, document signing	New (2024); being rolled out in libraries, HSMs, hybrid TLS/PKI	New standard (2024) but very conservative design (SPHINCS+);

				firmware, root keys
Main advantage	Smallest keys and signatures, fast signing	Very fast verification, universally supported	Quantum-safe with the best overall speed; fastest verification measured	Quantum-safe with security resting only on hash functions; 32-byte public key
Main disadvantage	Broken by Shor; needs good randomness/nonces	Broken by Shor; slow key generation, large keys for its security	Large public key and signature (about 3.8 KB per message)	Very slow signing and very large signatures

Table 7: Which algorithm would you choose?

Scenario	Chosen algorithm	Justification
Signing a firmware image for a device that stays in the field for 15 years	SLH-DSA	Must survive a future quantum computer; hash-only security is the most conservative, signing is done rarely at the vendor (840 ms is fine) and device verification is cheap (0.66 ms) with a 32 B key to store.
Authenticating a short sensor message on a very low-bandwidth link	ECDSA	Smallest overhead (71 B signature, 91 B key); a short-lived sensor reading needs no decades-long quantum resistance, while ML-DSA would add 2420 B per message.
Signing a TLS server certificate in 2026 on a general web server	ECDSA (P-256), moving to hybrid ECDSA + ML-DSA	Universally supported by browsers today with tiny handshakes; certificates live < 1 year, so authentication is not exposed to harvest-now-forge-later, and ML-DSA can be added as clients support it.
Root key of a certificate authority that must remain trusted for 30 years	SLH-DSA	Security relies only on hash functions (no new structured assumption); a root signs rarely so slow signing and 7.8 KB signatures are acceptable, and it must remain unforgeable well into the quantum era.
High-volume server that verifies millions of signatures per second	ML-DSA	Fastest verification measured (0.036 ms vs ECDSA 0.129 ms and RSA 0.059 ms) and quantum-safe; RSA-PSS verifies almost as fast but is not post-quantum.

7. Discussion questions

Conceptual

1. What are the three algorithms of a digital-signature scheme? Which key is used in which algorithm?

KeyGen() -> (PK, SK) creates the key pair and uses no key as input (only randomness; in this lab the ECDSA scalar is derived from the UID). Sign(SK, M) -> S uses the private key SK, known only to the signer. Verify(PK, M, S) -> accept/reject uses the public key PK, which anyone may hold. In hash-and-sign schemes (ECDSA, RSA-PSS) Sign and Verify work on h = H(M); ML-DSA and SLH-DSA take the whole message and hash it internally, which is why sign.py passes the document directly to them.

2. What is the difference between a digital signature and a MAC (for example HMAC)? Why does the signature allow non-repudiation?

A MAC uses one shared secret key for both creating and checking the tag, so it gives integrity and authentication only between parties that share the key; since the verifier can compute the same tag, a third party cannot tell who produced it. A signature uses an asymmetric key pair: only the holder of SK can produce S, while anyone with PK can verify. Because the verifier cannot create signatures, a valid signature is evidence to a judge or third party that the SK holder signed the message - the signer cannot later deny it (non-repudiation). MACs are, however, much faster and shorter.

3. Why is the private key never transferred in this experiment? What would happen if it were?

Verification needs only PK, and the security of the whole scheme rests on SK being known to the signer alone. Our channel (plain HTTP over sig-net) is readable by anyone on the network - the tcpdump capture shows the file contents in clear. If SK were sent, any eavesdropper (Mallory) could sign arbitrary documents such as "Transfer Rs.999999" that verify as PASS; integrity, authentication and non-repudiation would all be lost and the key pair would have to be revoked. This is why outbox/ contains only public files and keys/ is never served.

4. Why did post-quantum cryptography come into the picture? Explain Shor's algorithm's effect on RSA and ECDSA, and why a signature that is valid today may be dangerous to rely on in 20 years.

RSA is secure because factoring n = p·q is hard, and ECDSA because computing d from Q = d·G (the elliptic-curve discrete logarithm) is hard. Shor's algorithm on a large fault-tolerant quantum computer solves both problems in polynomial time, so an attacker could compute the private key directly from the public key and forge any signature; increasing key sizes does not help. Grover's algorithm only gives a quadratic speed-up against hash functions, and no efficient quantum attack is known on lattice problems - hence ML-DSA (lattices) and SLH-DSA (hashes), standardised by NIST in August 2024. Signatures that must stay trustworthy for decades (firmware, root certificates, legal archives, land records) are the concern: once a quantum computer exists, an attacker who recovers the old RSA/ECDSA key can create new signatures that are indistinguishable from the genuine ones, so a verifier in 20 years can no longer tell a real 2026 contract from a forged one. Long-lived systems and their keys must therefore migrate before such a machine exists.

Analytical (measured data)

5. Which algorithm had the smallest and which the largest signature? Express the difference as a ratio and explain the cause.

Smallest: ECDSA, 71 B (DER encoding of two 32-byte integers r, s). Largest: SLH-DSA-SHA2-128s, 7856 B - a ratio of 110.6x (and 3.25x ML-DSA's 2420 B, 20.5x RSA's 384 B). ECDSA is compact because elliptic-curve groups give 128-bit security with 256-bit numbers. An SLH-DSA signature must contain a FORS few-time signature plus a WOTS+ one-time signature and a Merkle authentication path for every one of the hypertree layers - many hash values of 16 bytes each. ML-DSA's signature is a vector of polynomials with 256 coefficients each (plus a hint), which is why it is in between.

6. Which algorithm was fastest to sign and which to verify? Why is SLH-DSA signing much slower than ML-DSA signing?

Fastest signing: ECDSA (0.0558 ms), closely followed by ML-DSA (0.1041 ms). Fastest verification: ML-DSA (0.0357 ms), followed by RSA-PSS (0.0589 ms, small exponent e = 65537) and ECDSA (0.1291 ms). SLH-DSA signing took 839.8 ms, about 8,067 times ML-DSA. ML-DSA signing is a few polynomial multiplications (NTT) with occasional rejection-sampling retries. SLH-DSA is stateless, so each signature must regenerate, from the secret seed, the parts of a huge hypertree of hash-based one-time keys: building the FORS trees and the WOTS+ chains and Merkle trees of every layer costs hundreds of thousands of SHA-256 calls. The 's' (small) parameter set makes this worse because it uses tall trees to shrink the signature; verification only recomputes one path, so it is fast (0.665 ms).

7. SLH-DSA has a tiny public key (32 bytes) but a large signature. ML-DSA has a larger public key but a smaller signature. Discuss the trade-off for a system that sends one public key but millions of signatures.

If the public key is sent once and signatures are sent N times, the cost is PK + N·|S|. For ML-DSA it is 1312 + 2420·N bytes and for SLH-DSA 32 + 7856·N bytes. SLH-DSA is cheaper only for N = 0; already with one signature ML-DSA's total (3732 B) is smaller than SLH-DSA's (7888 B), and for 1 million signatures ML-DSA needs about 2.42 GB against 7.86 GB for SLH-DSA (3.2x more bandwidth/storage) - the public-key size becomes irrelevant. Add to that SLH-DSA's ~8000x slower signing. So for high-volume signing ML-DSA is clearly preferable; SLH-DSA's tiny key matters where the key must be stored in very constrained places (ROM, fuse bits of a boot loader, a certificate chain) and few signatures are produced, e.g. firmware and root keys.

8. The public key was fetched over the same untrusted channel as the document. Explain how a man-in-the-middle could defeat the scheme and how certificates and a PKI solve this.

Mallory, sitting between sender and receiver (or impersonating the host name 'sender'), can replace all three files: she generates her own key pair, writes her own document, signs it with her SK and delivers her PK. The receiver verifies Mallory's signature with Mallory's key and gets PASS - verification only proves that the document matches the public key, not who owns the key. The fix is to authenticate the public key itself: a certificate authority signs a certificate binding PK_A to Alice's identity, the receiver checks the chain up to a root CA key that it obtained out of band (pre-installed trust store) and checks validity and revocation. Other ways are key pinning or comparing a key fingerprint over a separate trusted channel. In practice the transport is also protected by TLS, itself authenticated with certificates.

Application and evaluation

9. Is ML-DSA or SLH-DSA "better" than ECDSA? Answer using your data, and consider size, speed, maturity and quantum security. Is there a single winner?

There is no single winner. Size: ECDSA wins clearly - 221 B transmitted vs 3791 B for ML-DSA (17.2x) and 7947 B for SLH-DSA (36.0x). Speed: ML-DSA is as fast as or faster than ECDSA (key generation 0.59x, signing 1.87x, verification 0.28x of ECDSA), while SLH-DSA signs ~15,050x slower. Maturity: ECDSA has decades of analysis and universal support; the PQ standards are from 2024 and implementations are still being hardened. Quantum security: only ML-DSA and SLH-DSA survive Shor; SLH-DSA has the most conservative assumption. So ECDSA is best today where size matters and data is short-lived; ML-DSA is the best general-purpose PQ replacement; SLH-DSA is the choice for rarely-signed, long-lived roots of trust - and hybrids bridge the transition.

10. Why do many experts recommend hybrid signatures (a classical plus a post-quantum signature) during the transition period?

A hybrid signature is valid only if both component signatures verify, so it stays secure as long as at least one of the two algorithms is unbroken. The PQ schemes are new: cryptanalysis or implementation bugs (side channels, incorrect rejection sampling) may still be found, and some PQ candidates (e.g. Rainbow, SIKE) were broken shortly before or during standardisation. The classical scheme is well understood but falls to a future quantum computer. Combining them protects against both risks, satisfies compliance rules that still require FIPS-186 algorithms, and lets systems interoperate with old verifiers during migration. The cost is the sum of both sizes and times (e.g. ECDSA + ML-DSA: 2491 B of signatures).

11. Does the Docker network affect the cryptographic measurements? How did you keep Docker, the network and file I/O out of your timings, and what other factors can still influence them?

No - the network and Docker only affect the transfer, not the sign/verify computation. bench.py generates the 1024-byte message in memory, loads/creates keys before the timed region and wraps only the single library call (generate / sign / verify) between two time.perf_counter_ns() readings; there is no file access or network I/O in the loop, containers are already running, 5 warm-up calls are discarded, many repetitions give mean, std, min and max, and the last signature is verified so failures are not timed. Docker containers are ordinary processes using the host kernel, so CPU-bound code runs at native speed - the sender and receiver results agree within noise. Remaining influences: CPU frequency scaling / turbo and thermal state, other load on the host (this is a shared cloud VM - see the outliers in max_ms), scheduler pre-emption and cache effects, a container CPU limit (--cpus 1 changed little for these single-threaded operations), Python call overhead, library versions and build flags (AVX2 code paths in liboqs and OpenSSL), and the randomness of RSA prime generation.

Related questions referred to in the lab sheet:

Think box (Part F) - public key from the untrusted channel: see Question 8 - in practice the public key is delivered inside a certificate signed by a trusted CA (PKI), or pinned / checked by fingerprint out of band.

UID-derived keys (warning in Section 5): d is derived from a 10-digit UID, i.e. only about $10^{10} \approx 2^{33}$ possibilities, so an attacker can try every UID in seconds and recover the ECDSA private key. Our Test 4 shows the consequence directly: the receiver re-created the sender's private key just from the UID. Real private keys must come from a cryptographically secure random generator (os.urandom / the OS CSPRNG) with the full 256-bit entropy, and be stored protected (HSM, smart card or encrypted PKCS#8).

8. Final conclusion

A digital signature gives integrity and authentication because only the holder of the private key can produce a signature, while anyone holding the matching public key can check it; the private key never left the sender's container, and only the document, signature and public key crossed the sig-net network. Changing a single character of the document or a single bit of the signature made verification fail for all four algorithms, as did verifying with an unrelated public key. The ECDSA case was instructive: because its key was derived from my UID, the receiver regenerated the same key and verification passed, showing why low-entropy keys are dangerous. ECDSA had the smallest footprint (71-byte signature, 221 bytes transmitted), RSA-PSS verified quickly but needed slow key generation, ML-DSA-44 was as fast as or faster than ECDSA but transmitted 3,791 bytes, and SLH-DSA had a 32-byte public key but a 7,856-byte signature and about 840 ms signing. Post-quantum signatures are necessary because Shor's algorithm would break RSA and ECDSA, letting attackers forge signatures on documents that must stay trustworthy for decades; the price is larger keys and signatures and, for SLH-DSA, slower signing. For today's bandwidth-sensitive applications I would choose ECDSA; for general post-quantum use ML-DSA; and for long-term roots of trust and firmware SLH-DSA, whose security rests only on hash functions. I also learned to build one image, run isolated containers on a user-defined network, reach them by name, use bind mounts to keep results, and keep container overhead out of measurements.

(Word count: 249)

9. Source code

Dockerfile

```
FROM ubuntu:24.04
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y --no-install-recommends \
    python3 python3-pip python3-venv git cmake ninja-build build-essential \
    libssl-dev curl wget nano iproute2 iputils-ping tcpdump ca-certificates \
    && rm -rf /var/lib/apt/lists/*
RUN python3 -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir cryptography matplotlib pandas liboqs-python
# The first import makes liboqs-python download and build liboqs (a few minutes)
RUN python3 -c "import oqs; print(oqs.oqs_version())"
WORKDIR /lab
CMD ["sleep", "infinity"]
```

common.py

```
"""common.py - shared helpers for the digital-signature lab (sender and receiver).

Contents (Step D2):
  * STUDENT_UID read from the environment (must be exactly 10 digits)
  * ALGS: lab name -> library name / parameters
  * document text, ECDSA private scalar, benchmark message, tamper index (Section 5)
  * file-path helpers and a millisecond timing helper
"""
import hashlib
import os
import random
import sys
import time

# ---------------------------------------------------------------- UID
def get_uid():
    uid = os.environ.get("STUDENT_UID", "").strip()
    if not uid:
        sys.exit("ERROR: environment variable STUDENT_UID is not set "
                 "(start the container with -e STUDENT_UID=<your 10-digit UID>)")
    if not (uid.isdigit() and len(uid) == 10):
        sys.exit(f"ERROR: STUDENT_UID must be exactly 10 digits, got {uid!r}")
    return uid


UID = get_uid()

# ---------------------------------------------------------------- algorithms
# Strings for the PQ schemes are the exact names printed by Step D1 on this system.
ALGS = {
    "ecdsa":  {"family": "classical", "label": "ECDSA",   "curve": "SECP256R1", "hash": "SHA256"},
    "rsapss": {"family": "classical", "label": "RSA-PSS", "key_size": 3072, "e": 65537,
               "hash": "SHA256", "salt_length": 32},
    "mldsa":  {"family": "pq", "label": "ML-DSA",  "oqs_name": "ML-DSA-44"},
    "slhdsa": {"family": "pq", "label": "SLH-DSA", "oqs_name": "SLH_DSA_PURE_SHA2_128S"},
}
ALG_ORDER = ["ecdsa", "rsapss", "mldsa", "slhdsa"]


def check_alg(alg):
    if alg not in ALGS:
        sys.exit(f"ERROR: unknown algorithm {alg!r}; choose one of: {', '.join(ALG_ORDER)}")
    return alg


# ---------------------------------------------------------------- UID-derived values (Section 5)
```

```
# Order n of the P-256 group
P256_N = int("FFFFFFFF00000000FFFFFFFFFFFFFFFF"
             "BCE6FAADA7179E84F3B9CAC2FC632551", 16)


def document_text(uid=UID):
    """Line 1: Transfer Rs.<A> to Account <last 4 digits of UID>, A = 1000 + (UID mod 9000)
       Line 2: Issued by UID: <UID>"""
    amount = 1000 + (int(uid) % 9000)
    return f"Transfer Rs.{amount} to Account {uid[-4:]}\nIssued by UID: {uid}\n"


def ecdsa_private_scalar(uid=UID):
    """d = ( int( SHA-256( UID || "CNS-DSIG-ECDSA-KEY" ) ) mod (n - 1) ) + 1"""
    digest = hashlib.sha256((uid + "CNS-DSIG-ECDSA-KEY").encode()).digest()
    return (int.from_bytes(digest, "big") % (P256_N - 1)) + 1


def bench_message(uid=UID):
    """1024 bytes produced by random.Random(int(UID)).randbytes(1024)"""
    return random.Random(int(uid)).randbytes(1024)


def tamper_index(length, uid=UID):
    """index = UID mod (length of document or signature in bytes)"""
    return int(uid) % length


# ---------------------------------------------------------------- file paths
KEYS_DIR, OUTBOX_DIR, INBOX_DIR = "keys", "outbox", "inbox"


def private_key_path(alg):
    return os.path.join(KEYS_DIR, f"{alg}_private.key")


def public_key_path(alg, folder=OUTBOX_DIR):
    return os.path.join(folder, f"{alg}_public.key")


def signature_path(alg, folder=OUTBOX_DIR):
    return os.path.join(folder, f"{alg}.sig")


def document_path(folder=OUTBOX_DIR):
    return os.path.join(folder, "document.txt")


def read_bytes(path):
    with open(path, "rb") as f:
        return f.read()


def write_bytes(path, data):
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    with open(path, "wb") as f:
        f.write(data)


# ---------------------------------------------------------------- timing helper
def timed_ms(func, *args, **kwargs):
    """Run func once; return (result, elapsed milliseconds) measured with perf_counter_ns."""
    t0 = time.perf_counter_ns()
    result = func(*args, **kwargs)
    t1 = time.perf_counter_ns()
    return result, (t1 - t0) / 1e6
```

keygen.py

```
"""keygen.py <alg> - generate the key pair of the chosen algorithm (Step D3).

Private key -> keys/<alg>_private.key   (never leaves the container)
Public key  -> outbox/<alg>_public.key
"""
import sys

import oqs
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import ec, rsa

from common import (ALGS, check_alg, ecdsa_private_scalar, private_key_path,
                    public_key_path, timed_ms, write_bytes)


def _der_pair(private_key):
    sk = private_key.private_bytes(encoding=serialization.Encoding.DER,
                                   format=serialization.PrivateFormat.PKCS8,
                                   encryption_algorithm=serialization.NoEncryption())
    pk = private_key.public_key().public_bytes(
        encoding=serialization.Encoding.DER,
        format=serialization.PublicFormat.SubjectPublicKeyInfo)
    return sk, pk


def generate(alg):
    """Return (private_key_bytes, public_key_bytes) for alg."""
    if alg == "ecdsa":
        # private key built from the UID-derived scalar d on curve P-256
        return _der_pair(ec.derive_private_key(ecdsa_private_scalar(), ec.SECP256R1()))
    if alg == "rsapss":
        p = ALGS["rsapss"]
        return _der_pair(rsa.generate_private_key(public_exponent=p["e"], key_size=p["key_size"]))
    # ML-DSA / SLH-DSA: raw public key and raw secret key from liboqs
    with oqs.Signature(ALGS[alg]["oqs_name"]) as signer:
        pk = signer.generate_keypair()
        sk = signer.export_secret_key()
    return sk, pk


def main():
    if len(sys.argv) != 2:
        sys.exit("usage: python3 keygen.py <ecdsa|rsapss|mldsa|slhdsa>")
    alg = check_alg(sys.argv[1])
    (sk, pk), ms = timed_ms(generate, alg)
    write_bytes(private_key_path(alg), sk)
    write_bytes(public_key_path(alg), pk)
    name = ALGS[alg].get("oqs_name", ALGS[alg]["label"])
    print(f"[keygen] algorithm      : {alg} ({name})")
    print(f"[keygen] private key    : {private_key_path(alg):28s} {len(sk):6d} bytes")
    print(f"[keygen] public key     : {public_key_path(alg):28s} {len(pk):6d} bytes")
    print(f"[keygen] generation time: {ms:.3f} ms")


if __name__ == "__main__":
    main()
```

sign.py

```
"""sign.py <alg> - sign the UID document (Part E).

(1) create outbox/document.txt from the UID if it does not exist, (2) read it as bytes,
(3) load the private key, (4) sign, (5) write outbox/<alg>.sig, (6) print the signature size.
"""
import os
```

```
import sys

import oqs
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import ec, padding
from cryptography.hazmat.primitives.serialization import load_der_private_key

from common import (ALGS, check_alg, document_path, document_text, private_key_path,
                    read_bytes, signature_path, timed_ms, write_bytes)


def load_private(alg, sk_bytes):
    """Classical: cryptography key object. PQ: liboqs signer holding the secret key."""
    if ALGS[alg]["family"] == "classical":
        return load_der_private_key(sk_bytes, password=None)
    return oqs.Signature(ALGS[alg]["oqs_name"], sk_bytes)


def sign_data(alg, key, data):
    if alg == "ecdsa":
        # SHA-256 with ECDSA, DER-encoded signature
        return key.sign(data, ec.ECDSA(hashes.SHA256()))
    if alg == "rsapss":
        # PSS, MGF1(SHA-256), salt length = SHA-256 digest length (32), hash SHA-256
        return key.sign(data,
                        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                                    salt_length=ALGS["rsapss"]["salt_length"]),
                        hashes.SHA256())
    # ML-DSA / SLH-DSA: sign the whole document directly (no manual hashing)
    return key.sign(data)


def main():
    if len(sys.argv) != 2:
        sys.exit("usage: python3 sign.py <ecdsa|rsapss|mldsa|slhdsa>")
    alg = check_alg(sys.argv[1])

    doc = document_path()
    if not os.path.exists(doc):
        write_bytes(doc, document_text().encode())
        print(f"[sign] created {doc}")
    data = read_bytes(doc)

    key = load_private(alg, read_bytes(private_key_path(alg)))
    sig, ms = timed_ms(sign_data, alg, key, data)
    write_bytes(signature_path(alg), sig)

    print(f"[sign] algorithm     : {alg}")
    print(f"[sign] document      : {doc} ({len(data)} bytes)")
    print(f"[sign] signature     : {signature_path(alg)} ({len(sig)} bytes)")
    print(f"[sign] signing time  : {ms:.3f} ms")


if __name__ == "__main__":
    main()
```

verify.py

```
"""verify.py <alg> [document] [public_key] [signature] - verify a signature (Part G).

Defaults: inbox/document.txt, inbox/<alg>_public.key, inbox/<alg>.sig
Uses exactly the same parameters as sign.py; prints PASS or FAIL and the verification time.
"""
import sys

import oqs
from cryptography.exceptions import InvalidSignature
```

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import ec, padding
from cryptography.hazmat.primitives.serialization import load_der_public_key

from common import (ALGS, INBOX_DIR, check_alg, document_path, public_key_path,
                    read_bytes, signature_path, timed_ms)


def load_public(alg, pk_bytes):
    """Classical: cryptography public-key object. PQ: (liboqs verifier, raw public key)."""
    if ALGS[alg]["family"] == "classical":
        return load_der_public_key(pk_bytes)
    return oqs.Signature(ALGS[alg]["oqs_name"]), pk_bytes


def verify_data(alg, key, data, sig):
    """Return True if the signature is valid, False otherwise."""
    if ALGS[alg]["family"] == "classical":
        try:
            if alg == "ecdsa":
                key.verify(sig, data, ec.ECDSA(hashes.SHA256()))
            else:
                key.verify(sig, data,
                           padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                                       salt_length=ALGS["rsapss"]["salt_length"]),
                           hashes.SHA256())
            return True
        except InvalidSignature:          # cryptography raises on failure
            return False
    verifier, pk = key
    return verifier.verify(data, sig, pk)  # liboqs returns a boolean


def main():
    if not 2 <= len(sys.argv) <= 5:
        sys.exit("usage: python3 verify.py <alg> [document] [public_key] [signature]")
    alg = check_alg(sys.argv[1])
    doc = sys.argv[2] if len(sys.argv) > 2 else document_path(INBOX_DIR)
    pub = sys.argv[3] if len(sys.argv) > 3 else public_key_path(alg, INBOX_DIR)
    sig_file = sys.argv[4] if len(sys.argv) > 4 else signature_path(alg, INBOX_DIR)

    data, sig = read_bytes(doc), read_bytes(sig_file)
    try:
        key = load_public(alg, read_bytes(pub))
        ok, ms = timed_ms(verify_data, alg, key, data, sig)
    except ValueError as exc:             # e.g. malformed key / wrong key length
        ok, ms = False, 0.0
        print(f"[verify] error: {exc}")

    print(f"[verify] algorithm : {alg}")
    print(f"[verify] document  : {doc}")
    print(f"[verify] public key: {pub}")
    print(f"[verify] signature : {sig_file}")
    print(f"[verify] time      : {ms:.3f} ms")
    print(f"[verify] RESULT    : {'PASS (VALID)' if ok else 'FAIL (INVALID)'}")
    sys.exit(0 if ok else 1)


if __name__ == "__main__":
    main()
```

tamper.py

```
"""tamper.py <doc|sig> <file> - Mallory modifies one character/bit (Part H).

index = UID mod (file length); the byte at that index is XORed with 0x01 (exactly one bit,
hence exactly one character, changes). The result is saved as <file>.tampered; the original
```

```
file is never overwritten.
"""
import sys

from common import read_bytes, tamper_index, write_bytes


def main():
    if len(sys.argv) != 3 or sys.argv[1] not in ("doc", "sig"):
        sys.exit("usage: python3 tamper.py <doc|sig> <file>")
    kind, path = sys.argv[1], sys.argv[2]
    data = bytearray(read_bytes(path))
    idx = tamper_index(len(data))
    old = data[idx]
    data[idx] ^= 0x01
    out = path + ".tampered"
    write_bytes(out, bytes(data))

    print(f"[tamper] file      : {path} ({len(data)} bytes)")
    print(f"[tamper] index     : UID mod {len(data)} = {idx}")
    print(f"[tamper] byte      : 0x{old:02x} -> 0x{data[idx]:02x} (bit 0 flipped)")
    if kind == "doc":
        print(f"[tamper] character : {chr(old)!r} -> {chr(data[idx])!r}")
        print("[tamper] original text:")
        print(read_bytes(path).decode(errors="replace"), end="")
        print("[tamper] modified text:")
        print(bytes(data).decode(errors="replace"), end="")
    print(f"[tamper] written   : {out}")


if __name__ == "__main__":
    main()
```

bench.py

```
"""bench.py [output.csv] - benchmark key generation, signing and verification (Part I).

Only the cryptographic call is inside the timed region (no Docker start-up, file I/O or
network). Per algorithm and operation: 5 un-recorded warm-up runs, then key generation x20,
signing x100, verification x100, each call timed individually with time.perf_counter_ns().
The message is the 1024-byte UID-seeded message from Section 5.
"""
import csv
import platform
import socket
import statistics
import sys
import time

from common import ALG_ORDER, ALGS, UID, bench_message
from keygen import generate
from sign import load_private, sign_data
from verify import load_public, verify_data

WARMUP = 5
RUNS = {"keygen": 20, "sign": 100, "verify": 100}


def measure(func, runs):
    for _ in range(WARMUP):                      # warm-up, not recorded
        func()
    times, result = [], None
    for _ in range(runs):
        t0 = time.perf_counter_ns()
        result = func()
        t1 = time.perf_counter_ns()
        times.append((t1 - t0) / 1e6)            # ns -> ms
```

```
    return times, result


def stats(times):
    return {"mean_ms": statistics.mean(times), "std_ms": statistics.stdev(times),
            "min_ms": min(times), "max_ms": max(times)}


def main():
    out = sys.argv[1] if len(sys.argv) > 1 else "results.csv"
    msg = bench_message()
    assert len(msg) == 1024
    print(f"[bench] UID={UID} host={socket.gethostname()} python={platform.python_version()}")
    print(f"[bench] message: 1024 bytes, sha256-prefix={__import__('hashlib').sha256(msg).hexdigest()[:16]}")

    rows = []
    for alg in ALG_ORDER:
        # key generation (the key pair produced last is used for signing/verification)
        kg_times, (sk, pk) = measure(lambda: generate(alg), RUNS["keygen"])
        priv, pub = load_private(alg, sk), load_public(alg, pk)

        sg_times, sig = measure(lambda: sign_data(alg, priv, msg), RUNS["sign"])
        vf_times, ok = measure(lambda: verify_data(alg, pub, msg, sig), RUNS["verify"])

        # the signature from the last signing loop must verify, otherwise we timed failures
        if not (ok and verify_data(alg, pub, msg, sig)):
            sys.exit(f"ERROR: {alg} signature from the benchmark did not verify")

        for op, times in (("keygen", kg_times), ("sign", sg_times), ("verify", vf_times)):
            s = stats(times)
            rows.append({"uid": UID, "algorithm": alg, "operation": op, "runs": len(times),
                         **{k: f"{v:.4f}" for k, v in s.items()},
                         "public_key_bytes": len(pk), "private_key_bytes": len(sk),
                         "signature_bytes": len(sig)})
            print(f"[bench] {alg:7s} {op:7s} runs={len(times):3d} mean={s['mean_ms']:10.4f} ms "
                  f"std={s['std_ms']:9.4f} min={s['min_ms']:10.4f} max={s['max_ms']:10.4f}")
        print(f"[bench] {alg:7s} sizes: pk={len(pk)} B  sk={len(sk)} B  sig={len(sig)} B  "
              f"verify-check=PASS")

    fields = ["uid", "algorithm", "operation", "runs", "mean_ms", "std_ms", "min_ms", "max_ms",
              "public_key_bytes", "private_key_bytes", "signature_bytes"]
    with open(out, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=fields)
        w.writeheader()
        w.writerows(rows)
    print(f"[bench] written {out}")


if __name__ == "__main__":
    main()
```

plot.py

```
"""plot.py [results.csv] [document.txt] - draw the seven graphs of Section 15 into graphs/.

Every graph has a title, axis labels with units, a legend where required and the execution
environment in the caption (read from environment.txt if present). A logarithmic y-axis is
used wherever the values differ by orders of magnitude.
"""
import os
import platform
import sys

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
```

```
import pandas as pd

from common import ALG_ORDER, ALGS, UID

CSV = sys.argv[1] if len(sys.argv) > 1 else "results.csv"
DOC = sys.argv[2] if len(sys.argv) > 2 else os.path.join("outbox", "document.txt")
OUT = "graphs"
COLORS = {"ecdsa": "#2a6fb0", "rsapss": "#7a4fa3", "mldsa": "#2e8b57", "slhdsa": "#d0782a"}


def environment_caption():
    if os.path.exists("environment.txt"):
        with open("environment.txt") as f:
            env = " | ".join(line.strip() for line in f if line.strip())
    else:
        env = f"{platform.processor() or platform.machine()} | {platform.platform()}"
    return f"UID {UID} | Environment: {env}"


CAPTION = environment_caption()

df = pd.read_csv(CSV, dtype={"uid": str})
labels = [ALGS[a]["label"] for a in ALG_ORDER]
colors = [COLORS[a] for a in ALG_ORDER]
x = np.arange(len(ALG_ORDER))


def row(alg, op):
    return df[(df.algorithm == alg) & (df.operation == op)].iloc[0]


def sizes(col):
    return [int(row(a, "sign")[col]) for a in ALG_ORDER]


pk, sk, sig = sizes("public_key_bytes"), sizes("private_key_bytes"), sizes("signature_bytes")
doc = os.path.getsize(DOC)


def finish(fig, ax, name, title, ylabel, log=False, legend=False):
    ax.set_title(title, fontsize=12, fontweight="bold")
    ax.set_xlabel("Signature algorithm")
    ax.set_ylabel(ylabel)
    ax.set_xticks(x)
    ax.set_xticklabels(labels)
    if log:
        ax.set_yscale("log")
    ax.grid(axis="y", which="both", alpha=0.3)
    ax.set_axisbelow(True)
    if legend:
        ax.legend()
    fig.text(0.5, 0.01, CAPTION, ha="center", va="bottom", fontsize=6.5, wrap=True, color="#444")
    fig.tight_layout(rect=(0, 0.06, 1, 1))
    path = os.path.join(OUT, name)
    fig.savefig(path, dpi=150)
    plt.close(fig)
    print(f"[plot] wrote {path}")


def annotate(ax, bars, fmt="{:,.0f}", tops=None):
    for i, b in enumerate(bars):
        h = b.get_height()
        y = tops[i] if tops is not None else h
        ax.annotate(fmt.format(h), (b.get_x() + b.get_width() / 2, y), ha="center",
                    va="bottom", fontsize=8, xytext=(0, 2), textcoords="offset points")


def bar_with_err(op, name, title):
```

```
    mean = [row(a, op).mean_ms for a in ALG_ORDER]
    std = [row(a, op).std_ms for a in ALG_ORDER]
    fig, ax = plt.subplots(figsize=(8, 5))
    bars = ax.bar(x, mean, yerr=std, capsize=6, color=colors, edgecolor="black",
                  error_kw={"elinewidth": 1.2})
    annotate(ax, bars, "{:.3f}", tops=[m + sd for m, sd in zip(mean, std)])
    n = int(row(ALG_ORDER[0], op).runs)
    finish(fig, ax, name, title, f"Mean time per operation (ms, log scale), n={n}", log=True)


def main():
    os.makedirs(OUT, exist_ok=True)

    # Graph 1 - grouped bar: public and private key sizes
    fig, ax = plt.subplots(figsize=(8, 5))
    w = 0.38
    b1 = ax.bar(x - w / 2, pk, w, label="Public key", color="#2a6fb0", edgecolor="black")
    b2 = ax.bar(x + w / 2, sk, w, label="Private key", color="#d0782a", edgecolor="black")
    annotate(ax, b1); annotate(ax, b2)
    finish(fig, ax, "graph1_key_sizes.png", "Graph 1: Public- and private-key size",
           "Size (bytes, log scale)", log=True, legend=True)

    # Graph 2 - bar: signature size
    fig, ax = plt.subplots(figsize=(8, 5))
    annotate(ax, ax.bar(x, sig, color=colors, edgecolor="black"))
    finish(fig, ax, "graph2_signature_size.png", "Graph 2: Signature size",
           "Signature size (bytes, log scale)", log=True)

    # Graphs 3, 4 - bars with error bars (std)
    bar_with_err("sign", "graph3_signing_time.png",
                 "Graph 3: Average signing time (1024-byte message), error bar = std")
    bar_with_err("verify", "graph4_verification_time.png",
                 "Graph 4: Average verification time (1024-byte message), error bar = std")

    # Graph 5 - bar: key-generation time
    fig, ax = plt.subplots(figsize=(8, 5))
    kg = [row(a, "keygen").mean_ms for a in ALG_ORDER]
    annotate(ax, ax.bar(x, kg, color=colors, edgecolor="black"), "{:.3f}")
    finish(fig, ax, "graph5_keygen_time.png", "Graph 5: Average key-generation time",
           f"Mean time per key pair (ms, log scale), n={int(row('ecdsa', 'keygen').runs)}", log=True)

    # Graph 6 - stacked bar: document + signature + public key
    fig, ax = plt.subplots(figsize=(8, 5))
    d = np.array([doc] * len(ALG_ORDER)); s = np.array(sig); p = np.array(pk)
    ax.bar(x, d, label=f"Document ({doc} B)", color="#999999", edgecolor="black")
    ax.bar(x, s, bottom=d, label="Signature", color="#d0782a", edgecolor="black")
    bars = ax.bar(x, p, bottom=d + s, label="Public key", color="#2a6fb0", edgecolor="black")
    for b, tot in zip(bars, d + s + p):
        ax.annotate(f"{tot:,} B", (b.get_x() + b.get_width() / 2, tot), ha="center",
                    va="bottom", fontsize=9, xytext=(0, 2), textcoords="offset points")
    finish(fig, ax, "graph6_transmitted_bytes.png",
           "Graph 6: Total transmitted bytes = document + signature + public key",
           "Bytes sent over sig-net (linear scale)", legend=True)

    # Graph 7 (extra) - grouped bar relative to ECDSA
    fig, ax = plt.subplots(figsize=(9, 5))
    w = 0.26
    for i, (op, name, c) in enumerate((("keygen", "Key generation", "#2e8b57"),
                                       ("sign", "Signing", "#d0782a"),
                                       ("verify", "Verification", "#2a6fb0"))):
        base = row("ecdsa", op).mean_ms
        rel = [row(a, op).mean_ms / base for a in ALG_ORDER]
        annotate(ax, ax.bar(x + (i - 1) * w, rel, w, label=name, color=c, edgecolor="black"),
                 "{:.2f}")
    ax.axhline(1.0, color="black", linestyle="--", linewidth=1)
    finish(fig, ax, "graph7_relative_to_ecdsa.png",
           "Graph 7: Time relative to ECDSA (ECDSA = 1)",
           "Relative time (x ECDSA, log scale)", log=True, legend=True)
```

```
if __name__ == "__main__":
    main()
```

The same programs are copied to the receiver folder (cp ~/sig-lab/sender/.py ~/sig-lab/receiver/). The whole experiment can be repeated with run_experiment.sh, which also records every command transcript in logs/.*

10. results.csv files and UID

UID: **2024300032** (written in the first column of every result file).

Sender - results.csv (median run = results_run2.csv)

uid	algorithm	operation	runs	mean_ms	std_ms	min_ms	max_ms	pk_B	sk_B	sig_B
2024300032	ecdsa	keygen	20	0.0765	0.0187	0.0637	0.1487	91	138	71
2024300032	ecdsa	sign	100	0.0558	0.0225	0.0389	0.1707	91	138	71
2024300032	ecdsa	verify	100	0.1291	0.0261	0.1009	0.2590	91	138	71
2024300032	rsapss	keygen	20	215.7596	172.9181	34.3285	663.4801	422	1793	384
2024300032	rsapss	sign	100	2.2231	0.6104	1.8540	6.9156	422	1793	384
2024300032	rsapss	verify	100	0.0589	0.0108	0.0524	0.1047	422	1793	384
2024300032	mldsa	keygen	20	0.0454	0.0099	0.0395	0.0843	1312	2560	2420
2024300032	mldsa	sign	100	0.1041	0.0572	0.0481	0.2921	1312	2560	2420
2024300032	mldsa	verify	100	0.0357	0.0067	0.0331	0.0799	1312	2560	2420
2024300032	slhdsa	keygen	20	103.7940	13.2166	89.0875	129.4558	32	64	7856
2024300032	slhdsa	sign	100	839.7676	118.5879	663.9100	1226.1801	32	64	7856
2024300032	slhdsa	verify	100	0.6646	0.0175	0.6427	0.7716	32	64	7856

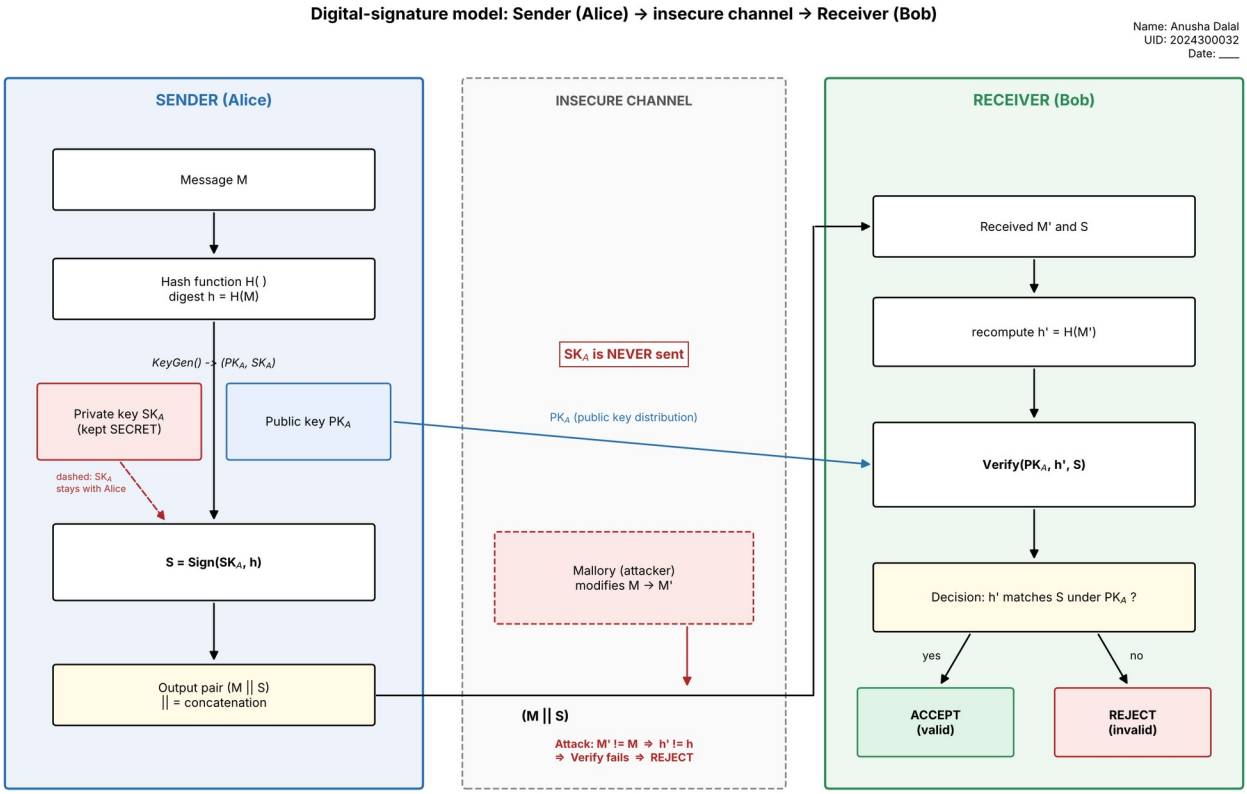
Receiver - results.csv

uid	algorithm	operation	runs	mean_ms	std_ms	min_ms	max_ms	pk_B	sk_B	sig_B
2024300032	ecdsa	keygen	20	0.0908	0.0204	0.0787	0.1586	91	138	71
2024300032	ecdsa	sign	100	0.0559	0.0136	0.0453	0.1295	91	138	71
2024300032	ecdsa	verify	100	0.1522	0.0246	0.1016	0.2678	91	138	71
2024300032	rsapss	keygen	20	258.2299	166.1175	75.0116	620.7739	422	1793	384
2024300032	rsapss	sign	100	2.6541	0.7561	1.8775	4.3344	422	1793	384
2024300032	rsapss	verify	100	0.0875	0.1650	0.0527	1.6344	422	1793	384
2024300032	mldsa	keygen	20	0.0471	0.0108	0.0391	0.0867	1312	2560	2420
2024300032	mldsa	sign	100	0.0954	0.0479	0.0474	0.2459	1312	2560	2420
2024300032	mldsa	verify	100	0.0365	0.0087	0.0329	0.1066	1312	2560	2420
2024300032	slhdsa	keygen	20	110.9689	17.3681	91.5047	150.8713	32	64	7856
2024300032	slhdsa	sign	100	786.0777	96.0449	663.8267	1360.1079	32	64	7856
2024300032	slhdsa	verify	100	0.6878	0.0325	0.6637	0.8916	32	64	7856

Header line of the CSV files:
uid,algorithm,operation,runs,mean_ms,std_ms,min_ms,max_ms,public_key_bytes,private_key_bytes,signature_bytes. Also submitted: results_run1.csv, results_run2.csv, results_run3.csv and results_cpus1.csv (sender).

Appendix A - Reference digital-signature diagram (for the hand-drawn Part A)

Computer-drawn reference containing all eight required elements. It does NOT replace the hand-drawn sheet required by Part A.



Symbols: M message, H hash function, h digest, S signature, SK private key, PK public key, || concatenation. REFERENCE ONLY - Part A must be drawn by hand.